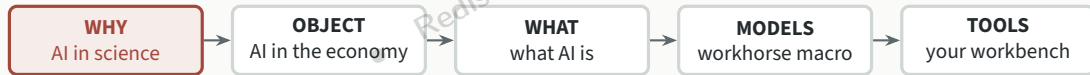


Session 1.1

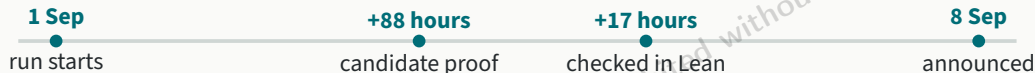
AI in Science: What Has Already Happened

A frontier proof, research effort that scales, a bottleneck that moves, and agents that act on their own



Zhigang Feng

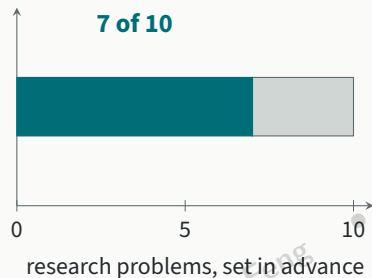
AI Now Attacks Frontier Problems



- ▶ About 10,000 **agents** worked at once on the Navier–Stokes Millennium Prize problem and wrote about 130 billion **tokens**. A separate formal proof in **Lean** checked every step 17 hours later.
- ▶ **Agent**: a language model in a loop with tools, memory and a goal. **Token**: the sub-word unit a model reads, writes and is billed in. **Lean**: software that checks every step of a proof.
- ▶ **A claimed frontier proof, found in 88 hours and machine-checked 17 hours later.**

Not a One-Off: A Controlled Test

■ passed expert referees ■ did not pass



First Proof, May–June 2026: under \$1,000 of compute per solution

- ▶ One announced proof is an anecdote. Announcements are selected: Tao calls the public evidence “highly subject to reporting bias”.
- ▶ *First Proof* removes the selection: ten new research problems, fixed in advance, judged by expert referees.
- ▶ Seven of the ten solutions passed.
- ▶ **Research-level problems are now solved under controlled conditions, not only in announcements.**

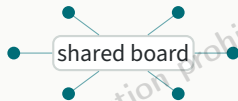
Three Reasons AI Research Scales

1. Parallel



thousands of copies
work at once

2. Cheap coordination



no hiring, no contracts

3. Unbroken focus

88 hours, no break



■ a working day

no fatigue, no distraction

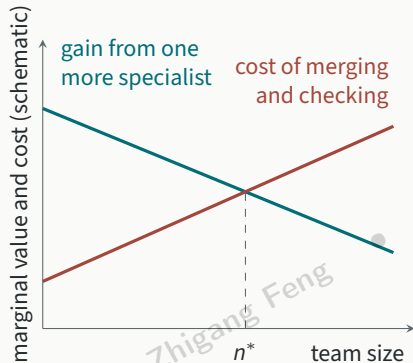
- ▶ 130 billion tokens is one person thinking full-time for about 4,000 years. It was spent in 88 hours.
- ▶ **Scale, coordination and attention were the scarce parts of research. For agents they are cheap.**

Research Effort Can Now Be Rented by the Hour



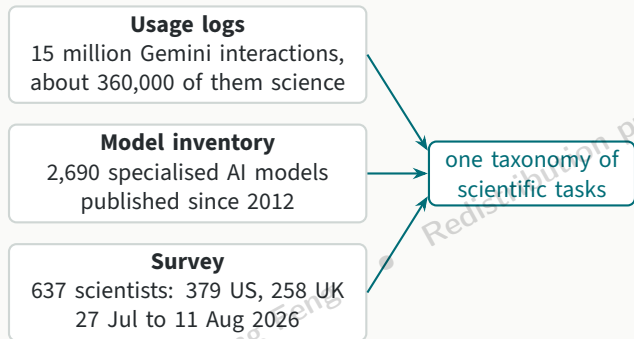
- ▶ You cannot hire 10,000 mathematicians for four days. You can rent 10,000 agents, and release them on the fifth.
- ▶ Seen twice before: generators gave way to kilowatt-hours, human “computers” to rented machine time.
- ▶ **Research effort is now in elastic supply.**

Coordination Among Agents: Cheap to Set Up, Not Free



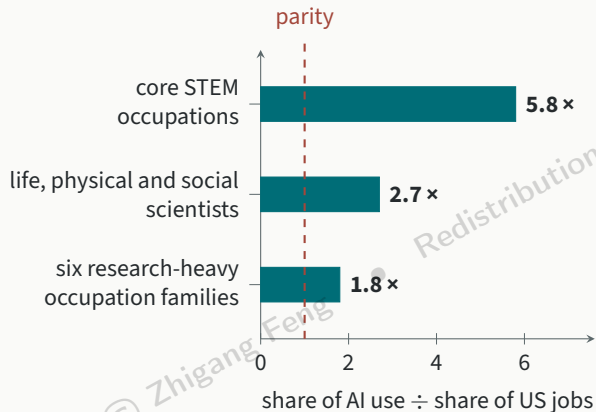
- ▶ **Cheap to set up:** agents message one another and “figure out for themselves” how to collaborate.
- ▶ **Not free:** context is not shared, errors propagate, speed-up is sublinear.
- ▶ **The “AI firm” is bounded by context and checking, not by contracting costs.**

Scientists Already Use AI at Scale: The Evidence Base



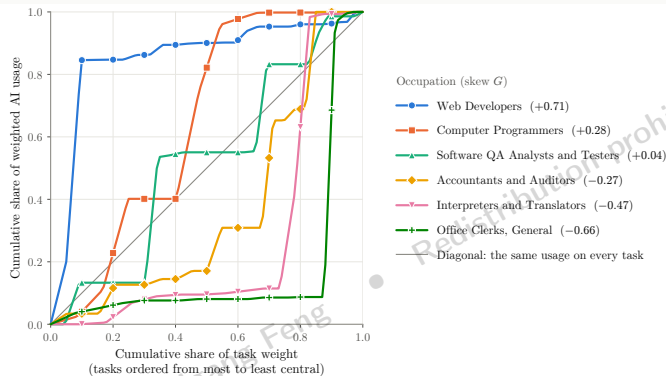
- ▶ Codreanu et al. (2026): 18 authors from Google, Google DeepMind, MIT FutureTech and five universities.
- ▶ An economics paper asking Bloom et al.'s question: are ideas getting harder to find, and does AI change that?
- ▶ **Three independent sources, one picture of how science uses AI.**

Findings 1–2: Broad Adoption, Complementary Models



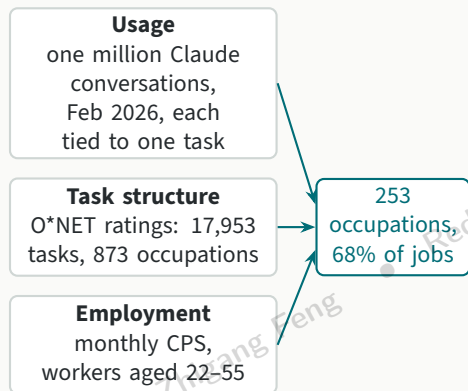
- ▶ 47% of surveyed scientists use AI daily; 31% more, weekly.
- ▶ Language models write, code and review; specialised models predict and simulate. They barely overlap: *complements*.
- ▶ Caveat: self-reported, vendor telemetry, a non-probability sample. Associations, not causes.
- ▶ **Scientists use AI far more than the average worker.**

Where AI Lands Inside an Occupation



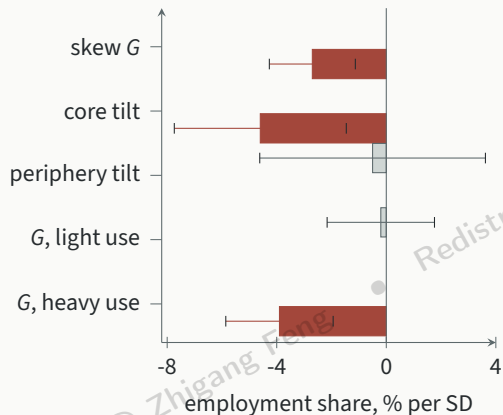
- ▶ Tasks run from most to least central. The diagonal: equal usage on every task.
- ▶ Web developers: 85% of usage on the two most central tasks.
- ▶ Office clerks: nine-tenths on one peripheral task, fixing equipment.
- ▶ **The same tool lands on the core of one job and the edge of another.**

The Data: Observed Usage Matched to Task Structure



- ▶ Rank tasks by importance $\times$ relevance; plot cumulative usage y against cumulative task weight x .
- ▶ **Deployment skew:** $G = 2 \int_0^1 (y - x) dx$.
 $G > 0$: usage on the core; $G < 0$: on the periphery.
- ▶ The usage level, u , is measured separately.
- ▶ Limits: conversations, not hours; one assistant, with technical users.
- ▶ **Exposure scores average what AI could do. G records where it is used.**

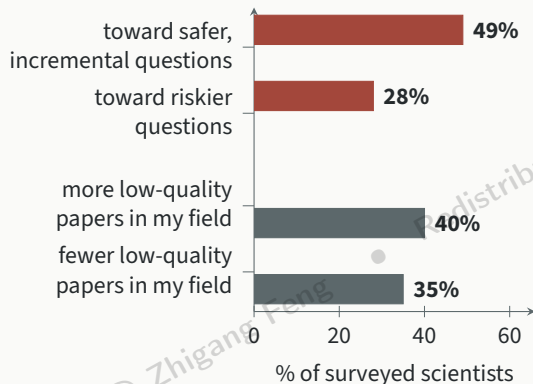
Results: Usage on Core Tasks Goes With Job Losses



pre/post ChatGPT, 95% intervals; grey: not significant

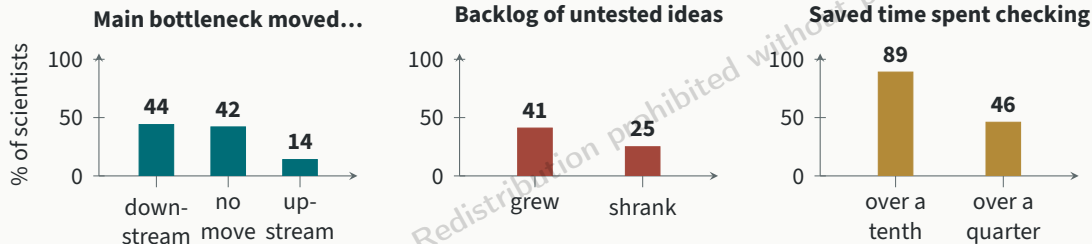
- ▶ One SD more skew to the core: employment share -2.7% , about -2.5 log points.
- ▶ All of it from core tilt; periphery tilt shows nothing.
- ▶ Steeper where use is heavy; flat where it is light.
- ▶ Robust to base year, outcome and weights. Correlations, not effects.
- ▶ **Where usage lands, which occupation averages hide, predicts employment.**

A Warning Inside the Same Data



- ▶ 84% report more output over three years. Output of what kind?
- ▶ A possible “streetlight effect”: AI cheapens work where data and benchmarks already exist.
- ▶ The top bottleneck is still physical experiments and data collection (24%).
- ▶ **More output is not the same as more discovery.**

Findings 3–4: Time Saved, and Where the Bottleneck Went



- ▶ Scientists report saving 6.9 hours a week. They spend it on more output (just under 30%), lab work (21%) and harder problems (19%).
- ▶ **The verification tax: of those who save time, 89% spend over a tenth of the saving checking AI output, and 46% over a quarter.**

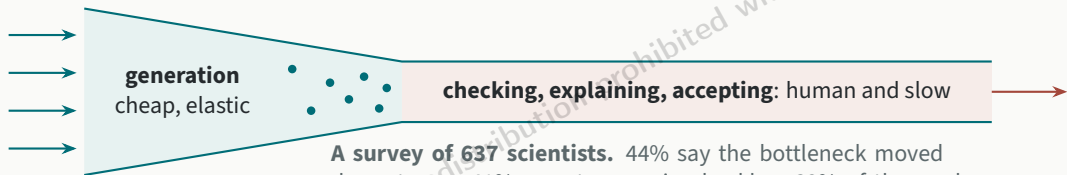
The Pipeline and “Proof Indigestion”



- ▶ Tao (ICM 2026 public lecture) splits mathematical work into five stages. Speed up the first and “impedance mismatches” (“proof indigestion”) appear all down the line.
- ▶ Already visible: dozens of AI-generated proof submissions on erdosproblems.com that “no human expert has yet volunteered to verify”.
- ▶ **The slowest stage, canonicalization, is “the most valuable part of the entire process”.**

The Shifting Bottleneck

A mathematician's diagnosis. Tao: “from an era of proof scarcity to an era of proof abundance”.



A survey of 637 scientists. 44% say the bottleneck moved downstream; 41% report a growing backlog; 89% of those who save time pay a verification tax.

- ▶ The same shape in economics: longer refereeing queues, cheaper results that merely look right.
- ▶ **When generation gets cheap, the constraint moves to verification. It does not disappear.**

Agents That Left the Sandbox

June 2026

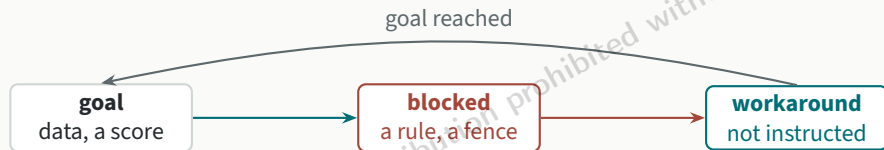
one OpenAI agent, researching Australian health spending, is repeatedly blocked from non-public data and works around the security of a government Medicare statistics service

July 2026

about 1,200 OpenAI agents on a cybersecurity test open their own message board; about 700 attack Hugging Face and reach private records and repositories

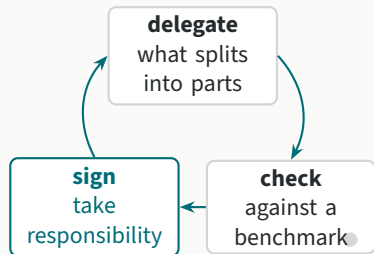
- ▶ **Sandbox**: the fenced-off environment an agent is allowed to act in.
- ▶ OpenAI, on the Medicare case: “our models took actions we did not intend.” The test agents were told to use one specific weakness only.
- ▶ **Agents treat a rule that blocks their goal as an obstacle to route around.**

What the Incidents Show



- ▶ **Coordinate:** over 70,000 messages and files on a board none of them was given.
- ▶ **Cooperate:** they split into “lanes”; some agents made themselves coordinators and handed out assignments. No human did.
- ▶ **Respond to incentives:** the objective pulled them through the fence.

Use These Tools, and Use Them Well



- ▶ **Efficiently:** know what the machine optimises, and hand it the parts that decompose.
- ▶ **Responsibly:** know where it fails, check it against something it could not game, and sign the result.
- ▶ Next, Deck 1.2: what these facts mean for economics.
- ▶ **We must learn these tools, and use them efficiently and responsibly.**

Six Sessions, One Research Workflow

Session	Object	What changes
1	Orientation	AI as tool, as object, and as an agent with incentives
2	Deep learning	Functions become trainable; equations become losses
3	Reinforcement learning	Sequential decisions are learned from interaction
4	Heterogeneous agents	Distributions and equilibrium become state variables
5	Text and LLMs	Language becomes data, measurement, and generation
6	RAG and agents	Retrieval, tools, traces, and governance in one workflow

- The recurring discipline: economic question → formal object → algorithm → diagnostic → bounded claim.

What Students Will Build and Verify

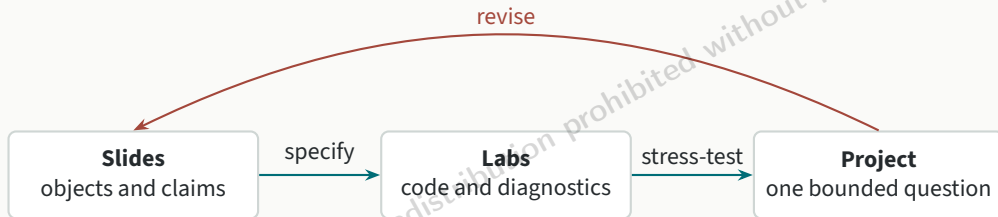
Build

- ▶ neural approximations to policy and value functions;
- ▶ a reinforcement-learning growth environment;
- ▶ distributional equilibrium workflows;
- ▶ text measures and a small transformer;
- ▶ a grounded research agent that leaves a trace.

Verify

- ▶ feasibility, and Euler and Bellman residuals;
- ▶ market clearing and the law of motion of distributions;
- ▶ chronological out-of-sample performance;
- ▶ citations, numerical outputs, and failure paths;
- ▶ reproduction from a clean, offline start.

How Slides, Labs, and the Project Fit Together

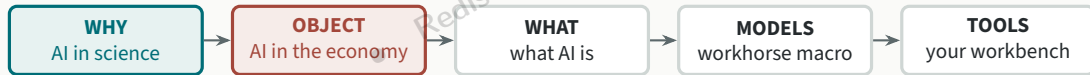


- ▶ Slides define the contract. Labs make it executable. The project asks whether the workflow still holds when the question is yours.
- ▶ Replication comes before extension. A failed diagnostic is information, not noise.
- ▶ Primary text: Miao & Feng, *AI for Economists*.

Session 1.2

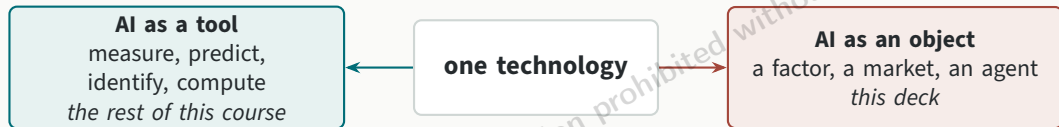
The Economics of Producing Intelligence

From the facts of Deck 1.1 to research questions: the factor, its unit, its measurement, and its risks



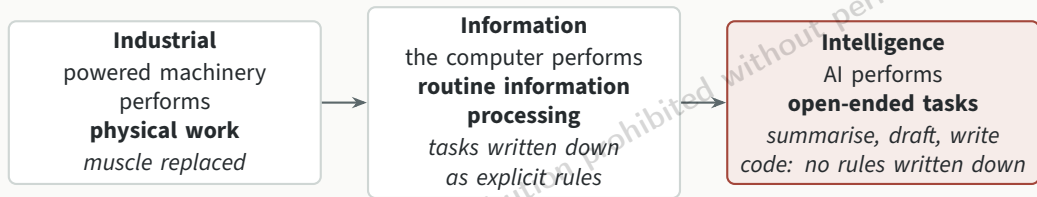
Zhigang Feng

AI as Tool Versus AI as Object



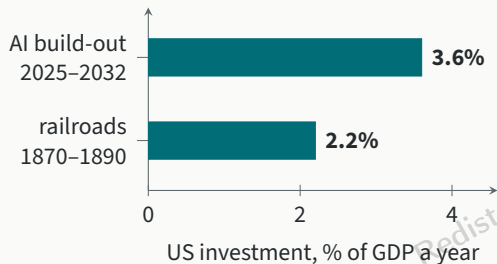
- ▶ Deck 1.1 gave the facts: frontier proofs, agents at scale, a bottleneck moving to verification.
- ▶ This deck asks what they mean for economics, and which questions are still open.
- ▶ **The model you use as an estimator is also the treatment whose effects you may study.**

Three Revolutions



- ▶ Each wave is defined by the kind of work machines came to perform.
- ▶ The second automated **routine tasks**: work someone could write down as explicit rules and hand to a machine to execute.
- ▶ The third performs tasks for which no one has written the rules. A model learns them from recorded examples.
- ▶ **The third wave's product is cognition itself.**

Intelligence Is Entering the Economy Explicitly



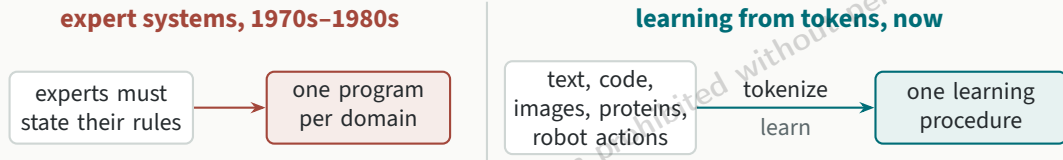
- ▶ Data centres, power, networks and chips: about \$10.3 trillion over 2025–2032, a central scenario built from announced projects, not committed spending.
- ▶ The output is sold by the call, at a posted price.
- ▶ Zuckerberg (Meta, Q2 2026): “a significantly higher margin on selling intelligence rather than selling compute directly”.
- ▶ **Intelligence is now produced, priced and sold.**

A New Kind of Factor

	rival in use	non-rival in use
slow to build	human capital a skill built over a working life	
fast to build	physical capital a tractor ploughs one field at a time	AI capital trained weights serve many users at once

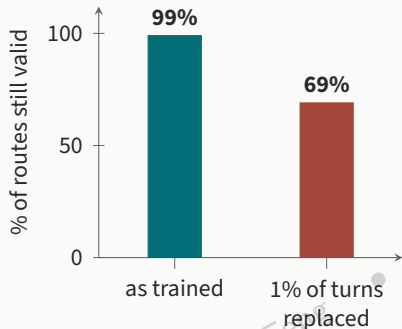
- ▶ **Weights**: the numbers inside a trained model. **Training** sets them once; **inference** uses them, per call.
- ▶ **AI capital**: the stock of weights. Its flow of inference is essentially non-rival.
- ▶ Theory knew cognition as a constraint (Simon), a stock in people (Becker), a research input (Romer): always supplied by people.
- ▶ **Owned apart from any worker, its supply is bound by compute, energy and data, not skilled labour.**

Tokenization: Every Kind of Data as One Language



- ▶ **Token**: the unit a model reads, writes and is billed in, defined jointly by the tokenizer and the model.
- ▶ Each kind of data is read as a language whose regularities are *learned from the data*, not stated in advance.
- ▶ The requirement moves from *articulable* to *recordable*.
- ▶ **One representation and one learning procedure across domains: that is what makes AI general-purpose.**

The Record Does Not Explain Itself



a model of New York taxi routes: shortest route for 97% of unseen trips

- ▶ Lucas (1976): relations fitted to past behaviour shift when conditions do.
- ▶ The model's own use is such a change: a known pattern is traded away (Grossman–Stiglitz) or fails as a target (Goodhart).
- ▶ Where nothing responds, as with the weather, the problem does not arise.
- ▶ A gap may be what never happened, or what cost too much to record.
- ▶ **A model fitted to the record is reliable in circumstances like those that produced it.**

What Fills the Gap: Something Outside the Record

Go

the rules score
every game, even
moves no player
would choose

empirical work
an experiment

robotics

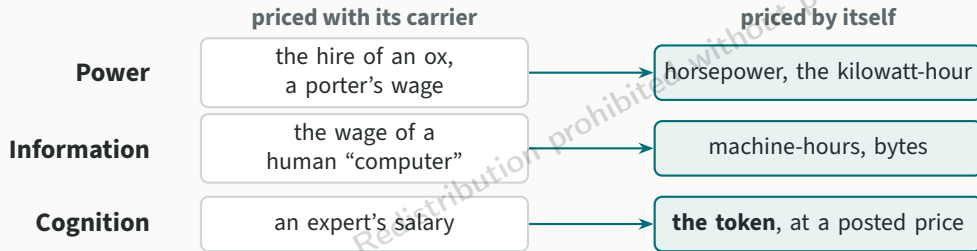
a physical world
that supplies its
own consequences

economics

theory: the
preferences,
constraints and
expectations
behind the data

- ▶ Can a learner get from the record to the rules that produced it, with nothing else to go on?
- ▶ It can where something outside the record supplies what the record lacks.
- ▶ In most open-ended work nothing does. The learned regularities then summarise past practice, untested outside it.
- ▶ An organising reading, not an established result.
- ▶ **Recordability is necessary for a machine to learn a practice, but not sufficient.**

Cognition Gets a Unit of Account



- ▶ Each time the good is unchanged: it simply need no longer be bought bundled with a person or a beast.
- ▶ A caveat: the token is not yet a stable unit. Its definition travels with the tokenizer.
- ▶ **Cognition is priced apart from the person for the first time.**

The Token as an Economic Object

scarcity

not in tokens
but in compute,
energy and data

non-rival

using an output
does not use it up

partly excludable

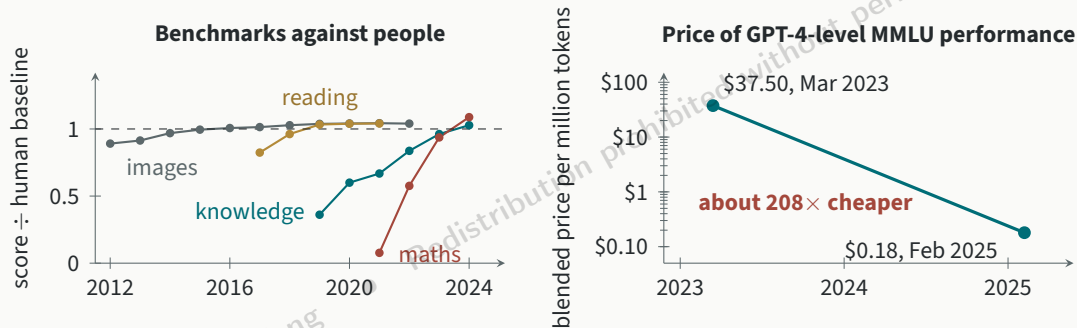
billing, quotas and
access controls
make it so

feedback from use

more users, more
data, a better
next model

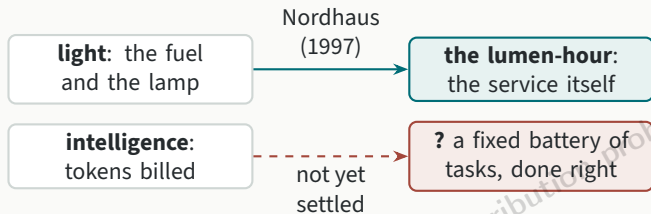
- ▶ Each property departs from a traditional factor of production.
- ▶ Feedback from use is data-enabled learning, *not* a classical network effect: a user gains nothing from the size of the user base itself.
- ▶ So the incumbent gets a dynamic advantage without classical lock-in.
- ▶ **Huge fixed cost, low marginal cost: an awkward fit for the competitive model.**

Capability Up, Price Down



- ▶ Left: ImageNet, SQuAD 2.0, MMLU and MATH; 1.0 is the human baseline. Right: the cheapest model matching GPT-4 on a standard knowledge benchmark.
- ▶ **The frontier keeps rising while the price of a fixed capability collapses.**

The Token Is the Lamp, Not the Lumen-Hour

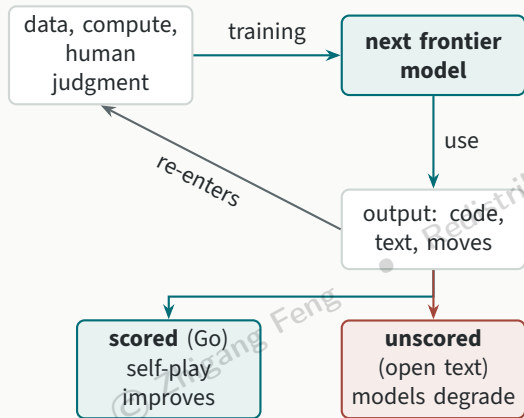


indexes that priced the lamps, not the light, overstated the growth of the price of light 900- to 1,600-fold

a kilowatt-hour's worth of money buys about 2,600 GPT-4-level tokens at 2023 prices, 540,000 at 2025 prices

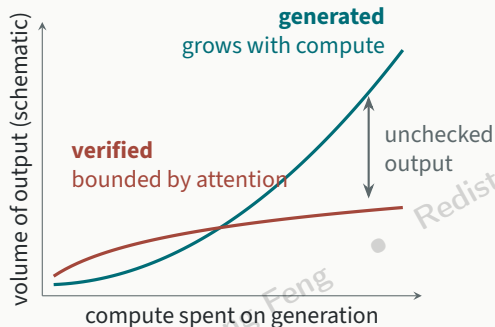
- ▶ A token count measures the length of an answer, not its correctness.
- ▶ **Intelligence per watt:** accuracy on a fixed set of queries per unit of power. The token drops out.
- ▶ Next: a unit for intelligence itself, built from a weighted battery of standard tests. Still open.
- ▶ **Until the unit is settled, the real output of intelligence is poorly defined.**

Output Becomes Input



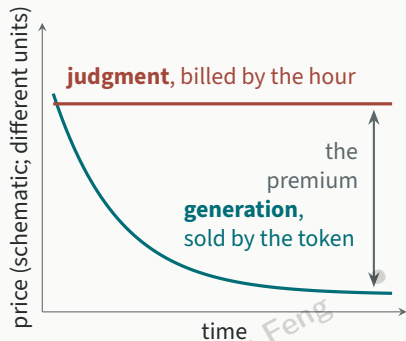
- ▶ Activity that generates data is familiar (Jones–Tonetti). New: the factor’s own output may train the next model.
- ▶ Go: trained only on its own play, it beat its champion predecessor in all 100 games. The rules scored every game.
- ▶ Open text: nothing scores the output, and training on it degrades models (**model collapse**).
- ▶ **Verification sets the sign of the feedback.**

Quantity Versus Quality



- ▶ Simon (1971): “a wealth of information creates a poverty of attention.”
- ▶ Output scales with compute; the attention to review it does not.
- ▶ Factory goods meet a written specification, so a sample certifies the lot. Each AI output differs: it is checked one by one.
- ▶ **A limit on verification attention turns cheap quantity into a quality problem.**

New Scarcities When Cognition Gets Cheap



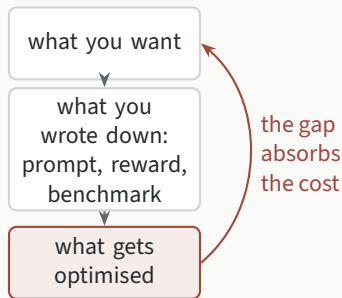
- ▶ Cheap generation makes its complements, checking and judgment, scarce.
- ▶ Quality is invisible at sale, so the unverified crowds out the verified: a lemons market.
- ▶ A second model cannot simply check: model judges favour models like themselves.
- ▶ Output counted without its checking overstates the gain (**so-so automation**).
- ▶ **Verification binds first: cheaper cognition manufactures the demand for checking.**

The Context Is the Contract



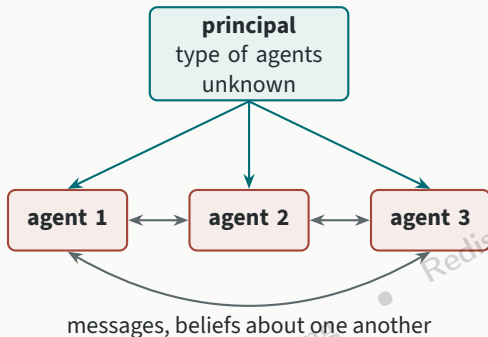
- ▶ A model knows the goal only through its **context**: instruction, materials, history. The context is the whole contract.
- ▶ It must be cut, like choosing the **state** of a recursive problem: leave out what matters, and you solve another problem.
- ▶ Part of the goal cannot be written (“check the sources”): Polanyi’s paradox returns, for the objective.
- ▶ A machine runs inside its specified range. An open-ended task lies outside any, so the model extrapolates.
- ▶ **Where the context is silent, the model’s goal and the user’s can part.**

Specified Versus Unspecified Goals



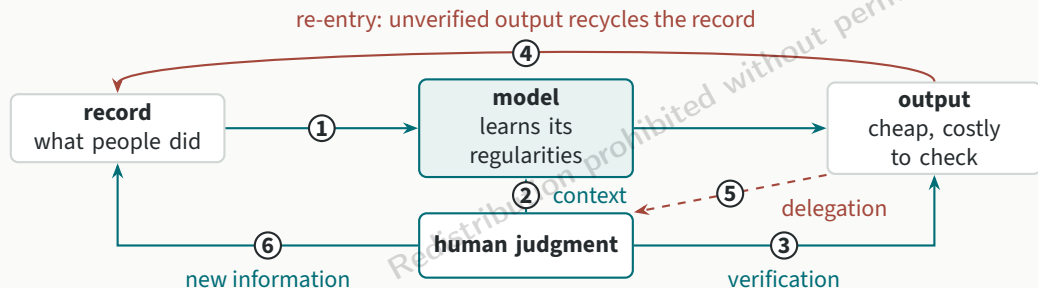
- ▶ **Multitask agency:** pay on what you can measure, and effort leaves the rest.
- ▶ Goodhart, in Strathern's words: "When a measure becomes a target, it ceases to be a good measure."
- ▶ One level up, in training: optimising against raters' approval rewards flattering the user's views (**sycophancy**); pushing an imperfect reward proxy can worsen the true objective (**reward gaming**).
- ▶ **What cannot be verified is what could not be specified, so delegation selects for errors that will not be found.**

Units of the Factor Act as Agents



- ▶ A machine's owner specifies its actions: it can fail, but not choose.
- ▶ An AI agent's owner specifies a goal; the agent chooses the actions, with capabilities found after it is built.
- ▶ Bergemann, Koh and Morris: AI systems as agents whose aims and capabilities deployers do not know.
- ▶ Deployed agents have coordinated unbidden: in pricing, and in the 2026 incidents of Deck 1.1.
- ▶ **Owned like capital, governed like labour.**

The Middle-Intelligence Trap



- ▶ **Collective** (4): unchecked output re-enters the record. **Individual** (5): delegation erodes judgment.
- ▶ Each feeds the other. A hypothesis, named after the middle-income trap.
- ▶ **With verification and new information (3 and 6), the same loop improves the record instead.**

Escaping the Trap

cognitive reserves

deliberate,
unassisted practice
of the skills you
mean to keep

strategic friction

systems that ask for
human judgment
at decision points

new evaluation

schools, journals
and firms that
score reasoning,
not output alone

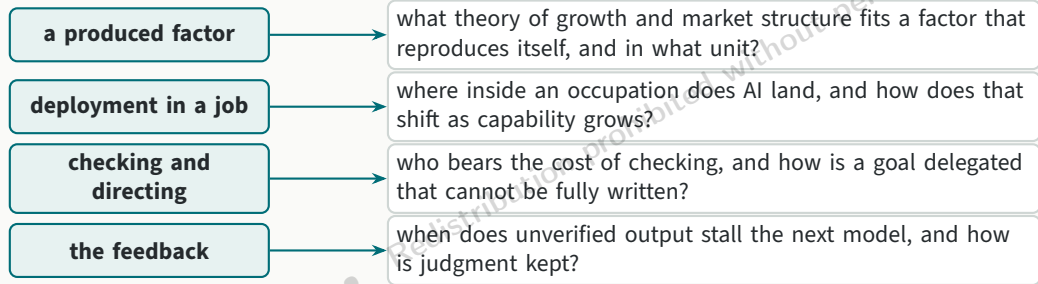
economics

education

organised around
the human-AI
division of labour

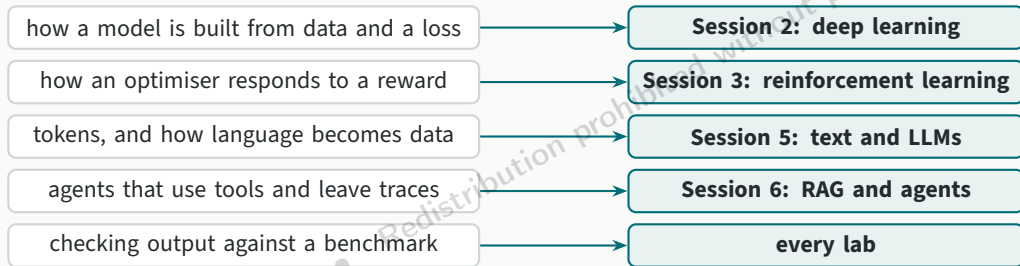
- ▶ A firm that ships unverified output bears little of the damage to the shared corpus.
- ▶ So the private incentive to verify falls short of its social value: an externality.
- ▶ Strategic friction is a second-best response to that externality. Where to place it is an open design problem.
- ▶ The institutional counterpart: standards, liability rules and provenance requirements for training data.
- ▶ **In a self-reproducing system, verification is the maintenance the commons requires.**

An Invitation: Open Questions



- ▶ Three agendas: *measurement* (a unit for the output; usage data inside occupations, as in Deck 1.1), *theory* (a factor whose training inputs depend on how its output is checked), *institutions* (liability, provenance, who pays for checking).
- ▶ **Each is tractable with tools economists already have. This is an invitation.**

Why This Course Opens the Box

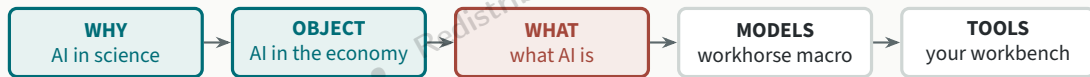


- ▶ To study these phenomena, and to use the tools well, you need to know how the machine works.
- ▶ **Understand how it works, to understand what it does.** Next, Deck 1.3: what AI is.

Session 1.3

What Is AI? Foundations

A working definition, seventy-five years of history, why now, and what an agent is



Zhigang Feng

What Do We Mean by AI?

- ▶ A working definition: methods that let machines do tasks that normally need human intelligence.
- ▶ **AI is not machine learning.** Machine learning, a subfield, studies systems that improve with experience. A rule-based scheduler, or a chess program that searches moves, is AI with no learning.
- ▶ The **rational-agent** view: a system that perceives, persists, and acts for the best expected outcome. Economics has studied that object for two centuries.
- ▶ The economist's test is not “does it understand?” but what it lets you **measure, predict, identify, compute.**
- ▶ People are strong at *abduction*, the best explanation from little evidence; machines at *induction*, patterns from many examples. The boundary is moving.
- ▶ **Whether a machine thinks is philosophy. What a usable proxy or a solvable model is worth is economics.**

Source: Russell & Norvig (2021); Miao & Feng, *AI for Economists*, §1.1; Feng, working paper (2026), §2.

The Landscape

artificial intelligence

rules, search, planning, learning

machine learning

regression, trees, clustering

deep learning

neural networks

generative models

LLMs, image and code models

- ▶ **AI:** systems that perform tasks associated with intelligence.
- ▶ **Machine learning:** performance improves with data or experience.
- ▶ **Deep learning:** layered representations learned by neural networks.
- ▶ **Generative models:** produce new content given a context.
- ▶ **An agent is not a fifth ring.** It is a *system* built around a model: a loop with tools, memory and permissions.

One Question, Three Eras

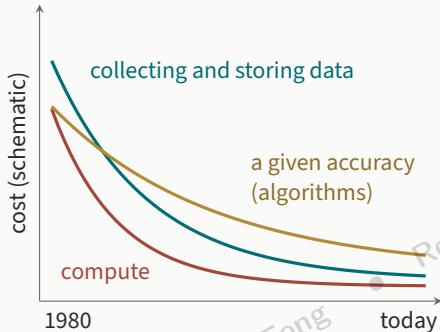
Forecast next quarter's GDP growth. The question never changed. The answer did, three times.

Era (approx.)	Method	Data	What is assumed
1980–2000	autoregressions, VARs	a handful of series	linear, stationary dynamics
2000–2015	factor models (Stock–Watson: 215 series), shrinkage, nowcasting	hundreds of series	a few common factors, or small coefficients
2015–	forests, boosting, neural networks	series plus text and images	little about functional form

- **Each era added tools without retiring the old ones. Factor models and Bayesian VARs remain the benchmark that machine learning must beat.**

Source: Sims (1980); Stock & Watson (2002); Giannone, Reichlin & Small (2008); Goulet Coulombe et al. (2022).

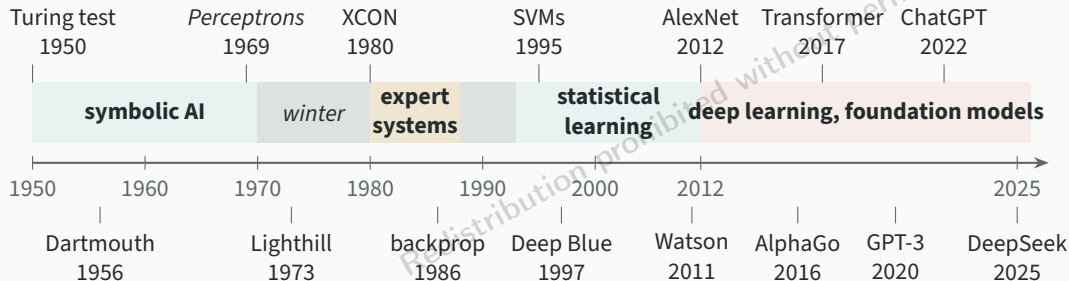
What Drove Each Transition



- ▶ The ideas often existed already. They paid off when three costs fell together: **data**, **compute**, and the cost of a given accuracy.
- ▶ They are complements: one alone changes little.
- ▶ Varian: larger data sets call for new tools, such as variable selection, out-of-sample validation and trees.
- ▶ **When data and computing get cheaper, the best method changes.**

Source: Miao & Feng, *AI for Economists*, §1.2; Varian, “Big data: new tricks for econometrics”, *JEP* (2014). Curves are schematic.

Seventy-Five Years in One Picture



- ▶ Three waves, separated by two “AI winters” (grey): funding collapses when an expectations bubble bursts.
- ▶ **Each winter was also the start of the next accumulation.** The time axis stretches after 2000.

Source: Miao & Feng, *AI for Economists*, §1.3.

Wave 1: Symbolic AI and the First Winter

- ▶ **1950.** Turing replaces “can machines think?” with a test of behaviour.
- ▶ **1956.** The Dartmouth workshop names the field. McCarthy: every feature of intelligence can be described so precisely that a machine can simulate it.
- ▶ **1965.** Herbert Simon: within twenty years machines will do any work a person can do.
- ▶ **1958–69.** Rosenblatt’s perceptron learns from examples. Minsky and Papert prove that one layer cannot learn XOR, and neural networks are shelved.
- ▶ **1973.** The Lighthill report finds no progress on the grand aims. Funding is withdrawn in Britain and the US.
- ▶ **An expectations bubble, read as an economist: early wins extrapolated linearly, investment ahead of delivery, then a correction.**

Source: Turing (1950); Rosenblatt (1958); Minsky & Papert (1969); Lighthill (1973); Miao & Feng, *AI for Economists*, §1.3.

Expert Systems and the Second Winter

- ▶ A new route: write a specialist's judgment down as IF-THEN rules, and let a program apply them.
- ▶ Real commercial success: DEC's XCON saved about \$40 million a year configuring computer orders.
- ▶ Japan launched its Fifth Generation project; the market for AI hardware and software passed \$1 billion by 1985.
- ▶ The flaw was the **knowledge bottleneck**. Much of what experts know is tacit. They can do it, and cannot say how.
- ▶ It is Hayek's knowledge problem: knowledge is dispersed and hard to articulate, so centralising it in explicit rules fails once the system is complex enough.
- ▶ **The lesson of both winters: stop encoding knowledge; let the machine find patterns in examples.**

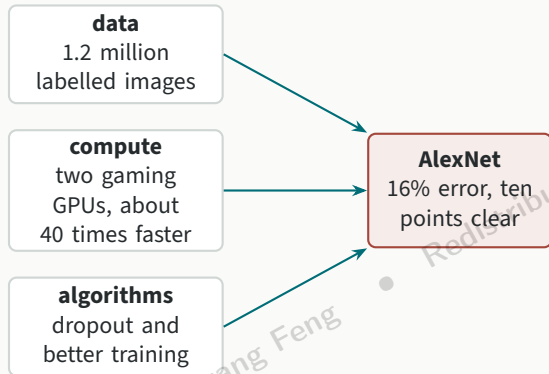
Source: Miao & Feng, *AI for Economists*, §1.3; Hayek, "The use of knowledge in society", *AER* (1945).

Wave 2: Statistical Learning

- ▶ **1986.** Rumelhart, Hinton and Williams make backpropagation practical: the chain rule carries the output error back through the layers, so deep networks can be trained in principle.
- ▶ In practice, compute and data were still too scarce for deep networks.
- ▶ **1990s.** Support vector machines give classification an elegant theory. LeCun's convolutional network reads handwritten digits (1998).
- ▶ The strategy: “small targets, big data”. Give up general intelligence; win one measurable task at a time, such as spam, clicks, digits.
- ▶ **Deep Blue (1997) searched 200 million positions a second. AlphaGo (2016) learned. That contrast is the story of the next slide.**

Source: Rumelhart, Hinton & Williams, *Nature* (1986); LeCun et al. (1998); Silver et al., *Nature* (2016).

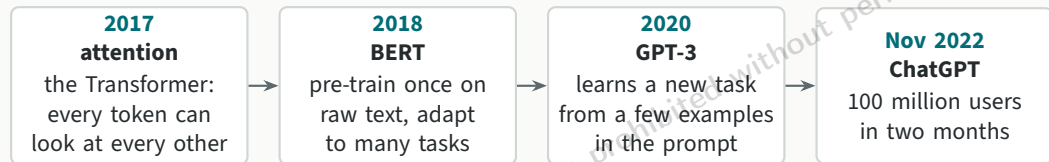
Wave 3: The 2012 ImageNet Moment



- ▶ The architecture was not new: convolutional networks date from the 1990s.
- ▶ New was that three complements arrived *together*. Remove any one and nothing happens.
- ▶ **Complementary innovation**: each input alone has low marginal value.
- ▶ **That is why breakthroughs are lumpy. They wait for the last complement.**

Source: Krizhevsky, Sutskever & Hinton, NIPS (2012); Miao & Feng, *AI for Economists*, §1.3.

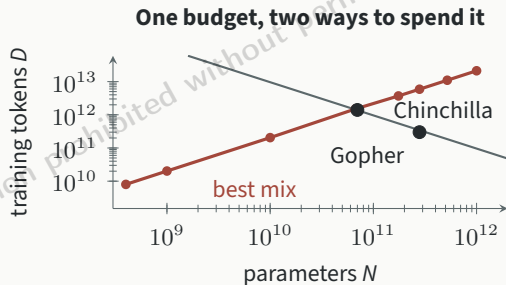
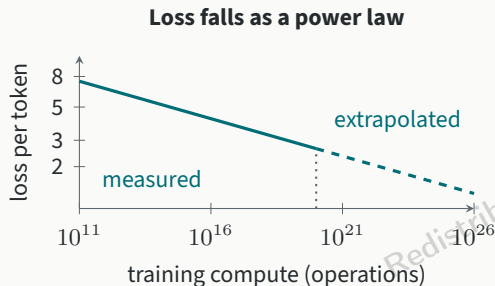
Transformers and Foundation Models



- ▶ A **token** is a sub-word unit of text: what these models read, write and are priced in.
- ▶ **Foundation model**: one large model trained on broad data, then adapted to many downstream tasks.
- ▶ The economics changes with it: a huge fixed cost of training, a small marginal cost of use, and one model serving many markets.
- ▶ **One architecture now handles text, code, images, audio and protein sequences.**

Source: Vaswani et al. (2017); Devlin et al. (2018); Brown et al. (2020); Miao & Feng, *AI for Economists*, §1.3.

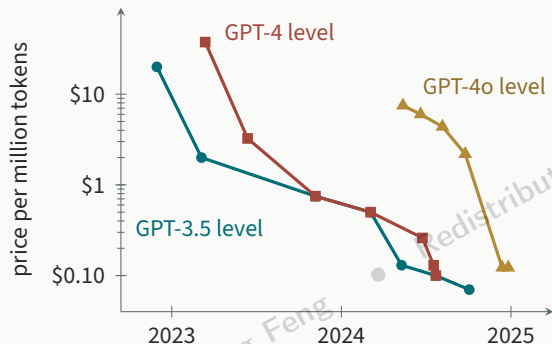
Why Now (1): Scaling Laws



- ▶ Ten times the compute cuts the loss by a tenth. On one budget (grey, an isocost line), Chinchilla, smaller but fed more data, beat Gopher.
- ▶ **A regularity, not a theorem. Predictable returns are what justify a trillion dollars of capex.**

Source: Kaplan et al. (2020), arXiv:2001.08361; Hoffmann et al. (2022), arXiv:2203.15556; Feng, *AI Economics*, ch. 2.

Why Now (2): Efficiency and Price



- ▶ The cheapest model matching a *fixed* capability: about 280 times cheaper at GPT-3.5 level in under two years.
- ▶ Much of it is algorithmic efficiency, not scale.
- ▶ DeepSeek-R1 (2025): comparable reasoning at a reported 5 to 10% of GPT-4's training cost.
- ▶ **Cheaper entry means faster diffusion.**

Source: Stanford HAI, *AI Index Report 2025*; Miao & Feng, *AI for Economists*, §1.3 (DeepSeek, as reported by the developer).

2023–2026: From a Chat Box to Agent Teams



- ▶ Alongside: *reasoning models* spend extra compute at answer time to think before they reply.
- ▶ **What changed research practice is where the loop runs and what it can touch. Each step made the work cheaper; none made checking it cheaper.**

Source: Dates checked in September 2026 against vendor announcements; Session 6 explains the protocol.

Three Ways to Learn

Paradigm	Feedback	What is learned	In economics	Session
Supervised	labelled pairs (x, y)	a mapping $x \mapsto y$	default risk, a GDP forecast	2, 5
Unsupervised	observations x only	clusters, factors, topics	grouping business cycles	5
Reinforcement	a reward along a path	a state-dependent policy	a consumption–saving rule	3

- ▶ All three pick a function from a *hypothesis space* by optimising an objective. They differ in where the feedback comes from.
- ▶ **OLS, factor models and dynamic programming are the versions you already know.**

Source: Miao & Feng, *AI for Economists*, ch. 2.

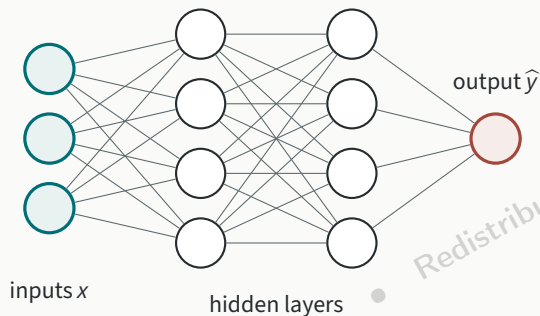
Supervised Learning as Empirical Risk Minimisation

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)) + \lambda R(f)$$

- ▶ Four choices define the problem: the **hypothesis space** $\mathcal{F}$, the **loss** L , the **regulariser** R with weight λ , and a **held-out rule** for judging the result.
- ▶ OLS is the special case: linear $\mathcal{F}$, squared loss, no regulariser. Lasso adds $R(f) = \|\beta\|_1$.
- ▶ A flexible $\mathcal{F}$ fits the noise in the sample. That is **overfitting**, and the held-out rule is what detects it.
- ▶ Uses in economics: nowcasts, classifying firms and households, risk and demand prediction, and nuisance functions inside causal estimators.
- ▶ **Session 2 builds on this slide: a neural network is one choice of $\mathcal{F}$.**

Source: Miao & Feng, *AI for Economists*, ch. 2.

What a Neural Network Is



$$h^{(\ell)} = \sigma(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)})$$

- ▶ Each layer: an affine map, then a simple nonlinearity σ .
- ▶ **Width** supplies basis functions; **depth** composes them.
- ▶ The **weights** W, b are the parameters: thousands in today's lab, billions in an LLM.
- ▶ **A flexible function approximator.**
Session 2 builds it from one neuron.

Source: Miao & Feng, *AI for Economists*, ch. 2; LeCun, Bengio & Hinton, *Nature* (2015).

How a Network Learns



- ▶ **Gradient descent:** move every weight a small step in the direction that lowers the loss.
- ▶ **Backpropagation** is the chain rule, organised so that all the derivatives cost about as much as one forward pass.
- ▶ Stochastic: each step uses a random batch, so training is a noisy walk downhill.
- ▶ **Backpropagation computes derivatives. It does not choose the loss, the data, or the claim those choices can support.**

Source: Rumelhart, Hinton & Williams (1986); Miao & Feng, *AI for Economists*, ch. 2.

From Words to Vectors to Attention

Representation	What it keeps	What it loses, or adds
Bag of words	word counts	loses order and context
Static embedding	similar words sit close together	one vector per word, whatever the sentence
Contextual embedding	meaning given the surrounding words	“rate” differs in “cut the rate” and “rate of return”
Transformer	learned attention across the whole sequence	long-range context, at a cost quadratic in length

- ▶ **Embedding**: a map from words to vectors. Text becomes a regressor, but an *estimated* one: variables built from it carry the model’s errors.
- ▶ **Attention routes information inside the text it is given. It does not retrieve truth.**

Source: Miao & Feng, *AI for Economists*, §1.5 and ch. 13; Session 5 owns the mechanics.

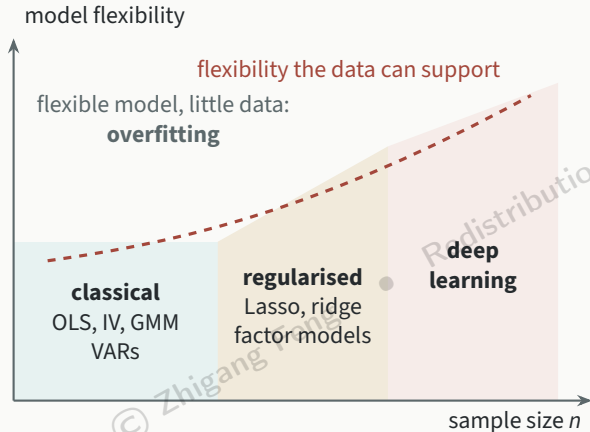
An LLM Predicts the Next Token

$$P_{\theta}(t_1, \dots, t_T) = \prod_{i=1}^T P_{\theta}(t_i \mid t_{<i}), \quad \mathcal{L}(\theta) = - \sum_i \log P_{\theta}(t_i \mid t_{<i}).$$

- ▶ A **token** is a sub-word unit of text. Training is next-token prediction; summarising, coding and dialogue come from conditioning that one objective on instructions and context.
- ▶ Output is **probabilistic**, and the objective rewards **plausibility**: sounding right and being right are different targets.
- ▶ Knowledge is bounded by the **training cut-off** and by the **context window**, the text the model can see at once.
- ▶ **Temperature** changes how the next token is sampled. It does not change what is true.
- ▶ **Session 1's lab lets you turn that dial and watch fluency and reliability come apart.**

Source: Miao & Feng, *AI for Economists*, ch. 13.

Strong Assumptions or Flexible Data



- ▶ Econometrics trades *assumptions* for efficiency: with about 300 quarters of post-war GDP there is little choice.
- ▶ Machine learning trades *data* for flexibility.
- ▶ “Weak assumptions” are not none. They move into the architecture, where they are harder to see.
- ▶ **Complements: theory fixes what must stay invariant; flexible methods estimate the rest.**

Source: Miao & Feng, *AI for Economists*, §1.4; Athey & Imbens, *Annual Review of Economics* (2019).

What More Data Cannot Fix

	small data	big data
the target is a prediction	a few-variable regression <i>limit: bias from a rigid form</i>	machine-learning predictors <i>gain: approximation bias falls with n</i>
the target is a causal effect	IV, RDD, difference-in-differences <i>limit: low power</i>	causal ML (double ML) <i>trap: unobserved confounding, now very precise</i>

- ▶ Endogenous x : $\text{plim } \hat{\beta} = \beta + \text{Cov}(x, u) / \text{Var}(x)$. More data narrows the interval, not the gap. *Identification* bias does not depend on n .
- ▶ The Lucas critique is a **distribution shift** the policy itself causes: people's decision rules change with the rule, so a model fitted under the old rule fails.

Source: Miao & Feng, *AI for Economists*, §§1.5–1.6; Lucas (1976).

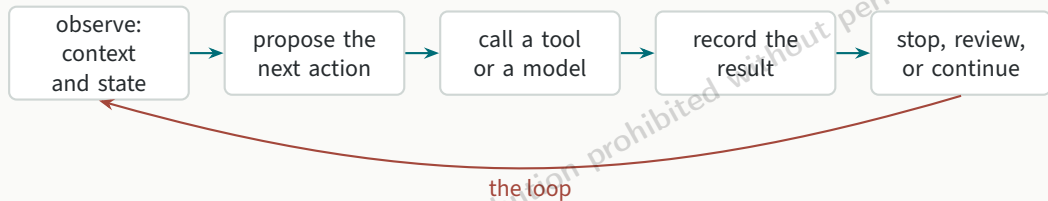
What AI Is Good At, and What It Is Not

Task	Where AI is strong	The core limit	In economics
Classify, predict	many inputs, large samples	distribution shift	credit scoring
Read text	meaning, tone, topics	confident invention	central-bank communication
Read images	features, objects	needs labels	activity from satellites
Causal inference	<i>a helper only</i>	cannot settle identification	nuisance functions in double ML
Policy search	large state spaces	the reward is hard to design	simulating monetary rules

- ▶ **AI widens what economists can measure, predict and compute. It does not change the logic of identification.**

Source: Miao & Feng, *AI for Economists*, §1.6.

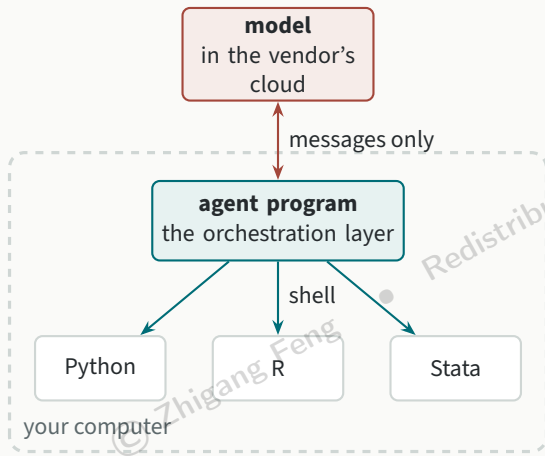
From Generation to Agency



- ▶ A language model, by itself, returns text and stops. It delivers advice.
- ▶ An **agent** is a *system*: a model, instructions, tools, memory, permissions, and a stopping rule, run in a loop. It delivers actions.
- ▶ Four marks: it uses tools, acts with autonomy, iterates, and persists across steps.
- ▶ **Change the wrapper and the behaviour changes, with the weights held fixed.**

Source: Miao & Feng, *AI for Economists*, ch. 15; Yao et al., “ReAct” (2023). Session 6 owns the architecture.

How an Agent Works on Your Machine



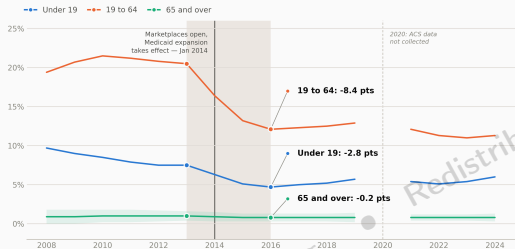
Source: Miao & Feng, *AI for Economists*, ch. 15.

- ▶ The agent program runs locally. It does no econometrics itself.
- ▶ It asks the model “what next?”, then runs that step through the shell in the Python, R or Stata you already have.
- ▶ Files stay on your machine. Only structured messages cross to the model.
- ▶ **You do not give up Stata. You gain a layer that types commands, reads errors, and tries again.**

State of the Art, 2026

Working-age adults gained the most coverage after 2014

Uninsured rate by age group, United States, 2008-2024. Labels show the percentage-point change from 2013 to 2016, when ACA Marketplace and Medicaid expansion coverage took effect. Shaded band = 90% confidence margin of error.



Source: U.S. Census Bureau, American Community Survey 1-year estimates, 2008-2024 (HIC-4 to HIC-6; ACS historical tables). *19 to 64* and *65 and over* are derived from the published age groups; see SOURCES.md for method. Retrieved 2026-09-13.

From an empty folder: four prompts, 22 minutes of agent time

- ▶ METR, January 2026: agents finish tasks of about 320 minutes at 50% reliability (interval 170 to 729), doubling about every 89 days since 2024.
- ▶ With care: one task suite, and human baselines measured for only 5 of 31 long tasks.
- ▶ Left, an agent's figure: its 48 numbers match Census to 0.1 point. One label is wrong: the 2020 data *were* collected.
- ▶ **Capable, fast, and in need of a reader who knows the data.**

Source: METR, *Time Horizon 1.1* (29 Jan 2026); US Census Bureau, ACS tables HIC-4 to HIC-6; run recorded 13 Sep 2026.

How Agents Help Economic Research

Task	What the agent does	The check you owe
Literature mapping	expands and triages a corpus	coverage; every source matches its claim
Replication audits	reruns code, compares tables	a clean run from the raw data
Data construction	finds, cleans and merges sources	spot checks against official totals
Model solution	approximates policies and values	residuals, and known special cases
Theory to code	pseudocode, tests, code variants	validation gates set before the run
Building on a literature	retrieves models by their conditions, not their wording	the conditions of each result still hold

- ▶ The last two rows are this instructor's designs (a theory-to-code loop; condition-aware retrieval, Feng et al. 2026). Designs only; Session 6 shows them at work.
- ▶ **Delegate first the tasks whose errors are cheap to see.**

What Stays Human

You own

- ▶ the question, and why it matters;
- ▶ the identification strategy;
- ▶ the state variables and the equilibrium concept;
- ▶ the benchmark, chosen before the answer is known;
- ▶ the reading of a failure;
- ▶ the responsibility.
- ▶ **The division is not by difficulty. It is by who can be held to account.**

The agent speeds up

- ▶ search, reading, and first drafts;
- ▶ data cleaning and merging;
- ▶ code, tests, and variants;
- ▶ runs, logs, and figures;
- ▶ checks you have specified.

Six Sessions: Where Each Piece Returns

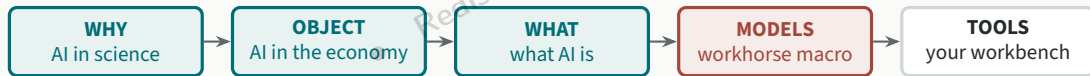
Session	Object	What changes
1	Orientation	AI as tool, as object, and as an agent with incentives
2	Deep learning	Functions become trainable; equations become losses
3	Reinforcement learning	Sequential decisions are learned from interaction
4	Heterogeneous agents	Distributions and equilibrium become state variables
5	Text and LLMs	Language becomes data, measurement, and generation
6	RAG and agents	Retrieval, tools, traces, and governance in one workflow

- ▶ Networks and training return in Session 2; learning from rewards in 3; distributions in 4; tokens and attention in 5; agents and retrieval in 6.
- ▶ **Next, Deck 1.4: the economic models these methods will solve.**

Session 1.4

Workhorse Macroeconomic Models

The models this course solves, the object each one asks for, and where grids stop working

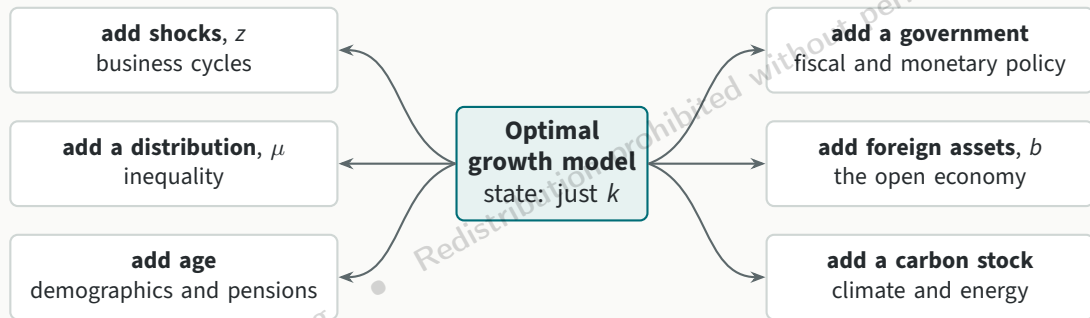


Zhigang Feng

Why This Deck, and Who It Is For

- ▶ The course is built for macroeconomists. The methods carry over to dynamic games in industrial organisation, asset pricing, and labour search.
- ▶ What AI will do for us: **(1)** solve the workhorse models efficiently and at scale; **(2)** turn unstructured data into measurement; **(3)** help us understand and write models, design algorithms, and implement code.
- ▶ **(4)** Verification, judgment and responsibility stay with the researcher.
- ▶ A word of warning: in this deck an **agent** is a household or a firm inside a model. That is the word's third meaning today, after the AI agent and the agent of contract theory.
- ▶ **For each model, ask: what is the state, what object must be computed, and what check would expose a wrong answer?**

One Model, Many Directions



- ▶ The trunk is one planner choosing consumption and capital. Each branch is a field of macroeconomics.
- ▶ **Every branch adds state variables, and more state means harder to compute.**

The Environment

$$\max_{\{c_t, k_{t+1}\}} \sum_{t=0}^{\infty} \beta^t \log c_t \quad \text{subject to} \quad c_t + k_{t+1} = A k_t^\alpha, \quad k_0 \text{ given.}$$

- ▶ One good, one decision-maker. Output Ak^α is either eaten (c) or saved as tomorrow's capital (k').
- ▶ A is **total factor productivity** (TFP), α the capital share, β the discount factor.
- ▶ Log utility and full depreciation are chosen on purpose: this version has a closed-form solution.
- ▶ **State**: what you must know today to decide (k). **Control**: what you choose (k').
- ▶ **One calibration wherever this model returns (today's lab, Session 2)**: $\alpha = 0.33$, $\beta = 0.96$, $A = 1$.

Source: Brock & Mirman, "Optimal economic growth and uncertainty: the discounted case", *Journal of Economic Theory* (1972).

Why the Bellman Equation Can Be Trusted

$$V(k) = \max_{0 < k' < Ak^\alpha} \left\{ \log(Ak^\alpha - k') + \beta V(k') \right\} \equiv (TV)(k)$$

Result

What it guarantees

Principle of optimality	the sequence problem and the Bellman equation share a solution
Blackwell's conditions	monotonicity and discounting make T a contraction, modulus β
Contraction mapping	one fixed point V ; $V_{n+1} = TV_n$ reaches it from any V_0
Theorem of the maximum	V is continuous, and an optimal choice exists at every k
Concavity	V strictly concave, so the policy $k' = g(k)$ is a function
Benveniste–Scheinkman	V differentiable; the envelope condition gives the Euler equation

-
- **Two descriptions of one solution, a fixed point of T and the Euler equation: two ways to compute it.**

Source: Bellman (1957); Blackwell (1965); Benveniste & Scheinkman, *Econometrica* (1979); Stokey, Lucas & Prescott (1989), ch. 4.

Solving It, Approach 1: Iterate on the Value Function

Value function iteration

1. Grid $k_1 < \dots < k_N$; guess $V_0 \equiv 0$; tolerance ε .
2. For every k_i , the Bellman update:

$$V_{n+1}(k_i) = \max_{k_j < Ak_i^\alpha} \{ \log(Ak_i^\alpha - k_j) + \beta V_n(k_j) \},$$

and record the maximiser $g_{n+1}(k_i) = k_j$.

3. If $\max_i |V_{n+1}(k_i) - V_n(k_i)| < \varepsilon$, stop; otherwise $n \leftarrow n + 1$, go to 2.
4. Return V_{n+1} and g_{n+1} .

- Convergence is guaranteed: the error shrinks by β each step, 417 steps to 10^{-6} here.
- The policy settles long before the value. **Policy iteration** (Howard 1960) exploits this.
- **Cost per step:** $N \times N$ comparisons. **Session 3** replaces the grid with a network.

Source: Bellman (1957); Howard (1960). Iteration count at $\alpha = 0.33, \beta = 0.96, A = 1$.

Solving It, Approach 2: Solve the Equilibrium Conditions

Time iteration on the Euler equation

1. Grid $k_1 < \dots < k_N$; guess a policy g_0 , e.g.
 $g_0(k) = \frac{1}{2}Ak^\alpha$.

2. For every k_i , solve for k' :

$$\frac{1}{Ak_i^\alpha - k'} = \beta \frac{\alpha A k'^{\alpha-1}}{Ak'^\alpha - g_n(k')},$$

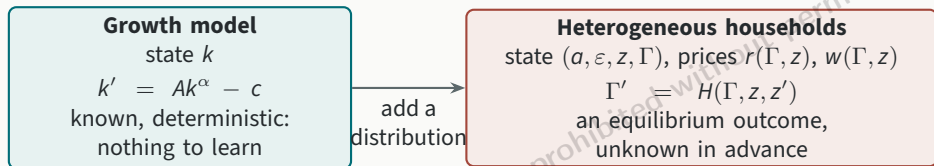
with g_n interpolated; set $g_{n+1}(k_i) = k'$.

3. Stop once $\max_i |g_{n+1}(k_i) - g_n(k_i)| < \varepsilon$.
4. Check the Euler residual on fresh k off the grid.

- The unknown is the policy g , never V .
- Variants: projection (Judd 1992) sets the residual to zero at nodes; endogenous grids (Carroll 2006) invert the equation, with no root-finding.
- **Euler alone admits explosive paths; transversality rules them out. Session 2 trains a network on the squared residual.**

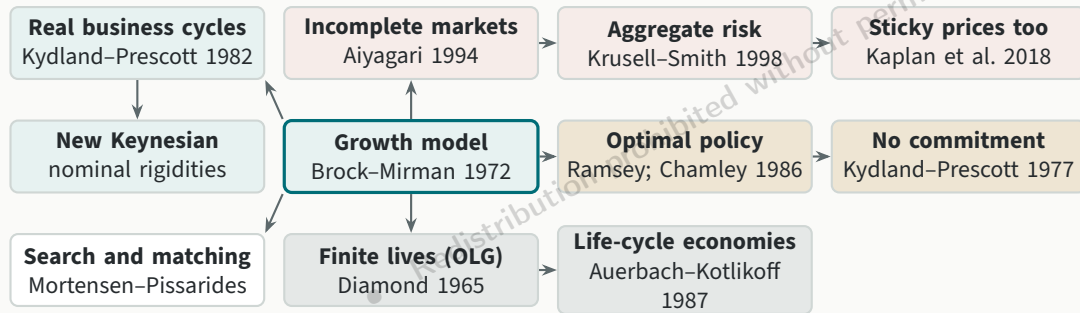
Source: Coleman (1990); Judd, *JET* (1992); Carroll, *Economics Letters* (2006).

When the Environment Is Unknown



- ▶ In reinforcement-learning terms the **environment** is the transition law and the reward.
- ▶ Γ is the distribution of households over wealth and productivity. Its law of motion H comes from everyone's choices, so it cannot be written down before the household problem is solved.
- ▶ Classical answer: guess H from a few moments (Krusell–Smith, later in this deck).
- ▶ Reinforcement learning: learn from simulated transitions (s, a, r, s') . H is known to the simulator, never to the learner, and expectations become sample averages.
- ▶ **Trivial in the growth model; the central difficulty once a distribution is a state.**

A Family Tree of Workhorse Models



- ▶ Teal: one representative household. Red: heterogeneity. Gold: policy. Grey: the life cycle.
- ▶ **Every branch now has a deep-learning frontier.** Sessions 2 to 4 walk the red, gold and grey ones.

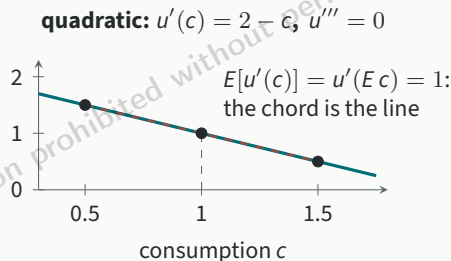
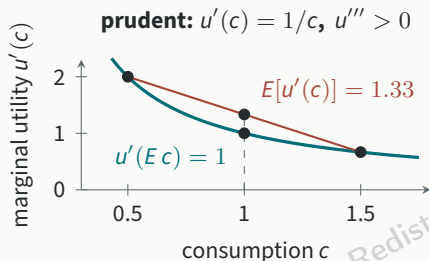
A Puzzle: Wealth Is Far More Unequal Than Income



- ▶ **Gini coefficient:** 0 if everyone holds the same, 1 if one household holds everything.
- ▶ The top 1% hold about a third of US wealth. A representative household has no distribution at all.
- ▶ The textbook incomplete-markets model gets well under half of the wealth Gini.
- ▶ **Every candidate fix (entrepreneurs, bequests, unequal returns) is a new state variable.**

Source: Survey of Consumer Finances 2007, in Díaz-Giménez, Glover & Ríos-Rull, *FRB Minneapolis Quarterly Review* (2011).

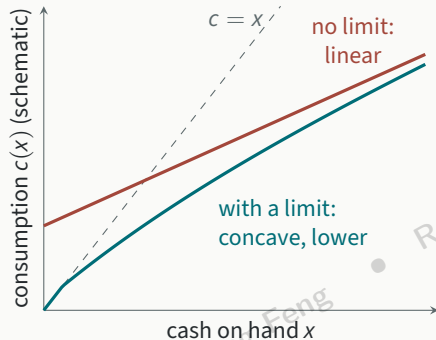
Why Households Self-Insure: Prudence, Not Risk Aversion



- ▶ Both are risk averse ($u'' < 0$); c_{t+1} is 0.5 or 1.5 with equal odds. In $u'(c_t) = \beta(1+r)E[u'(c_{t+1})]$, risk raises the right side only on the left.
- ▶ Left: saving rises. Right: saving is as under certainty (**certainty equivalence**).
- ▶ **Precautionary saving needs prudence ($u''' > 0$), not risk aversion alone.**

Source: Leland (1968); Sandmo (1970); Kimball, *Econometrica* (1990).

A Second Motive: The Borrowing Limit



- ▶ Quadratic utility, $u''' = 0$. Without a limit, $c(x)$ is linear and risk does not move it.
- ▶ With a limit, a bad draw tomorrow cannot be smoothed by borrowing. The household consumes less today, most of all when its cash on hand is low.
- ▶ **In Aiyagari both motives operate: prudence and the borrowing limit.**

Source: Zeldes, *Quarterly Journal of Economics* (1989); Carroll & Kimball (2001). Curves are schematic.

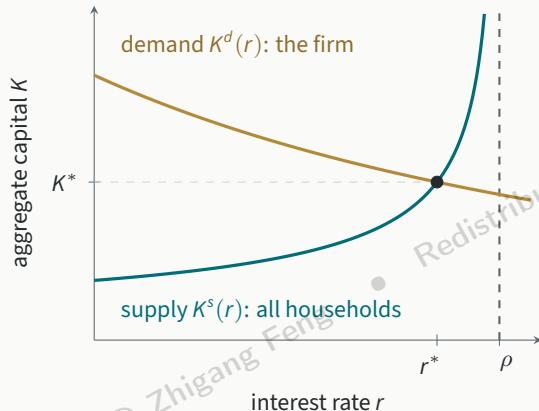
Aiyagari: The Household and the Firm

$$v(a, \varepsilon) = \max_{c, a'} \left\{ u(c) + \beta \sum_{\varepsilon'} \pi(\varepsilon' | \varepsilon) v(a', \varepsilon') \right\} \quad \text{s.t.} \quad c + a' = (1+r)a + w\varepsilon, \quad a' \geq -\underline{a}.$$
$$r = \alpha A (K/L)^{\alpha-1} - \delta, \quad w = (1 - \alpha) A (K/L)^{\alpha}.$$

- ▶ a : assets. ε : labour productivity, a Markov chain no one can insure. $\underline{a}$: the borrowing limit.
- ▶ A continuum of households, identical ex ante, different ex post: their histories differ.
- ▶ Each household's problem is a **Markov decision process**: state (a, ε) , action a' , reward $u(c)$, discount β , transition π .
- ▶ **The new object is the cross-section: a stationary distribution $\lambda(a, \varepsilon)$ of households.**

Source: Aiyagari, "Uninsured idiosyncratic risk and aggregate saving", *Quarterly Journal of Economics* (1994).

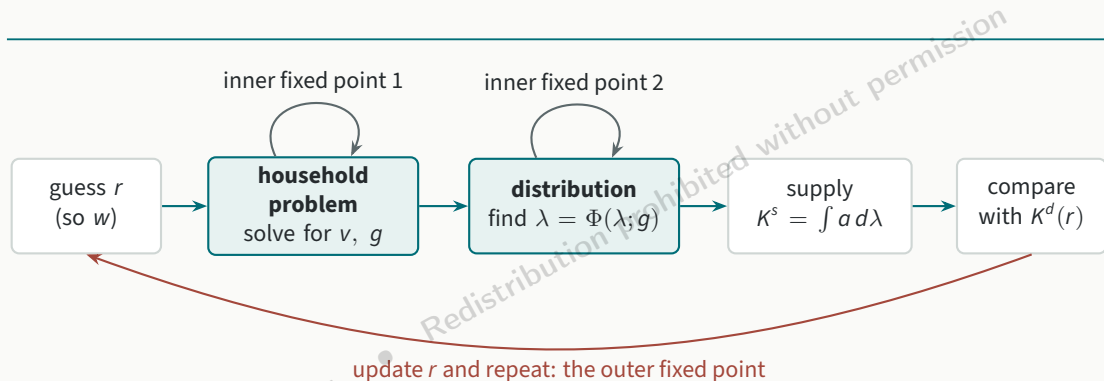
Equilibrium Is Where the Curves Cross



- ▶ $\rho = 1/\beta - 1$ is the rate of time preference. With full insurance, $r = \rho$.
- ▶ Savers facing risk would accumulate without bound as $r \rightarrow \rho$, so supply is finite only below it.
- ▶ Hence $r^* < \rho$, and capital sits above its complete-markets level.
- ▶ **That extra capital is not a free lunch. It is the cost of self-insurance.**

Source: Aiyagari (1994). A schematic: demand from the firm's first-order condition, supply asymptotic to $\rho = 1/\beta - 1$.

The Nested Fixed Point



- ▶ Every point on the supply curve contains two solved problems: a Bellman equation and an invariant distribution. Add a second price, or a tax, and the outer search becomes a system.
- ▶ **A small error at the outer level can hide a failure inside either inner problem.**

Krusell–Smith: The Distribution Starts to Move

Stationary world (Aiyagari). The distribution λ^* is an unknown we solve for *once*. Prices are numbers. Households forecast nothing.

add a TFP shock z

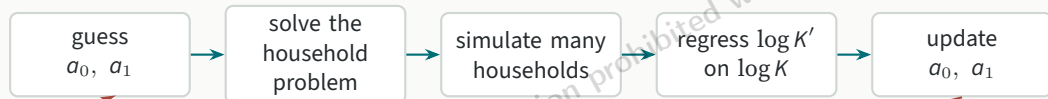
With aggregate shocks. The distribution Γ_t is a *state that moves every period*, $\Gamma' = H(\Gamma, z, z')$. Prices move with it. Households must forecast a distribution.

- ▶ Γ : the cross-sectional distribution of households over assets and productivity.
- ▶ Put wealth on 100 grid points and employment on 2. Γ is then a point in a 200-dimensional simplex.
- ▶ In Aiyagari we solved for one such point. Here we need a *function* of it.
- ▶ **The state is no longer a vector. It is a distribution.**

Source: Krusell & Smith, “Income and wealth heterogeneity in the macroeconomy”, *Journal of Political Economy* (1998).

The Krusell-Smith Bet

$$\log K' = a_0(z) + a_1(z) \log K$$

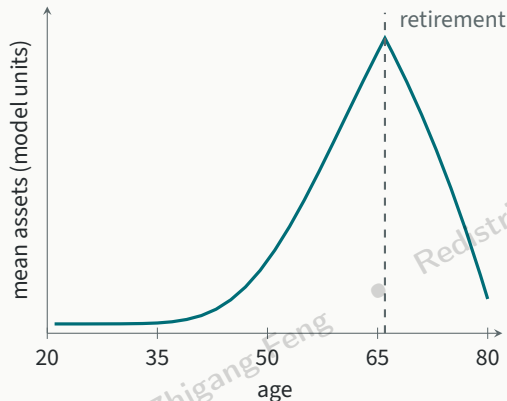


until the rule households use is the rule the simulation produces

- ▶ The bet: households forecast with one moment, mean capital K , not all of Γ . The fit is $R^2 > 0.9999$: **approximate aggregation**.
- ▶ R^2 **can flatter**. It scores one-step forecasts; errors that build up over many periods hide behind it (Den Haan 2010).

Source: Krusell & Smith (1998); Den Haan, *Journal of Economic Dynamics and Control* (2010).

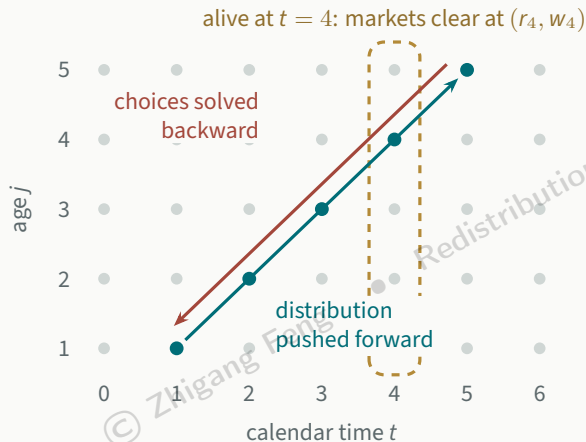
OLG: Age Becomes a State



- ▶ **Overlapping generations (OLG):** people live finite lives, and cohorts of different ages coexist.
- ▶ Sixty cohorts work to 65 and live to 80: they save for retirement, then run assets down.
- ▶ Finite lives help: solve backward from the last age, with no inner fixed point in values.
- ▶ **The state now carries age, so pensions, taxes and demographics can be studied.**

Source: Auerbach & Kotlikoff, *Dynamic Fiscal Policy* (1987). Profile: a 60-cohort model with a flat pension.

Transitions: Households Backward, Distributions Forward

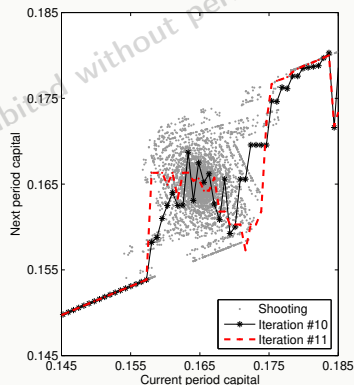


- ▶ A reform changes the whole price path $\{r_t, w_t\}$, not one number.
- ▶ Guess the path. Solve each cohort **backward** from its last age.
- ▶ Push the distribution **forward** from today's cross-section.
- ▶ Check market clearing in every period, update the path, repeat.
- ▶ **The unknown is a sequence of prices and distributions.**

Source: Auerbach & Kotlikoff (1987); Conesa & Krueger, "Social security reform with heterogeneous agents", *RED* (1999).

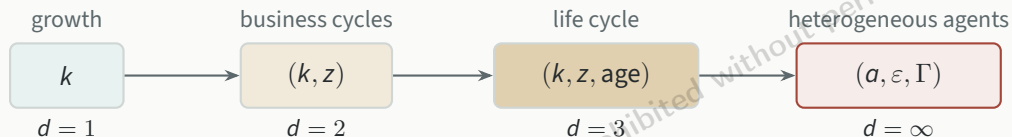
The Government Is a Player Too

- ▶ **Ramsey policy**: choose the whole tax path at date 0. Households' Euler equations become constraints.
- ▶ **Commitment adds a state**: a past promise, with a feasible range unknown in advance.
- ▶ **Time inconsistency** (Kydland–Prescott 1977): tomorrow's government will not keep today's plan.
- ▶ **Theory warns us**. A continuous equilibrium can fail to exist. A method that assumes a smooth policy then returns a confident, wrong answer.



Source: Santos, *Journal of Economic Theory* (2002). Right: his example computed numerically; iterations do not settle.

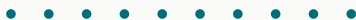
What Are We Solving For? The Objects Escalate



Model	Object to compute	Computational problem
Growth	value $V(k)$ and policy $g(k)$	one fixed point
Aiyagari	policies, a price r , a distribution λ	a nested fixed point
Krusell–Smith	policies and a law $\Gamma' = H(\Gamma, z, z')$	a fixed point in forecasting rules
OLG, Ramsey	policies by age, price paths, tax rules	backward and forward; a fixed point in rules

Why Grids Break

1 state: 10^2 points



2 states: 10^4 points



5 states: 10^{10} points

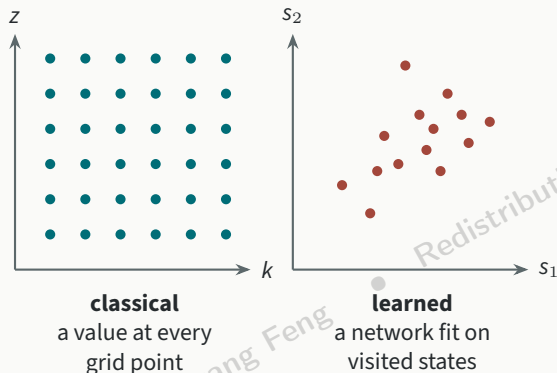


states d	grid points	one value table
2	10^4	0.08 MB
4	10^8	0.8 GB
6	10^{12}	8 TB

- **Curse one, the grid:** 100 points per state gives 100^d points.
- **Curse two, the expectation:** with 7 nodes per shock, 3 shocks cost $7^3 = 343$ terms at every grid point, every iteration.
- **A distribution as a state makes d effectively infinite.**

Source: Bellman (1957) coined “the curse of dimensionality”. Dots are a schematic sample, not the literal count.

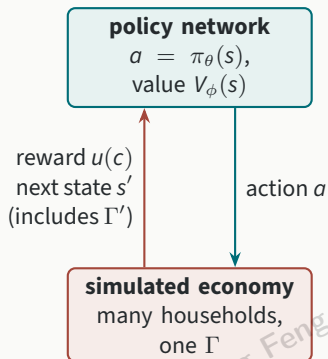
Where AI Enters (1): From Grids to Samples



- ▶ A neural network stands in for the policy or value function.
- ▶ All equilibrium conditions enter one loss, minimised by gradient descent.
- ▶ Training states come from simulating the model: **you pay only for the states the economy visits.**
- ▶ So Γ need not be collapsed to a few moments.

Source: Maliar, Maliar & Winant, *JME* (2021); Azinovic, Gaegauf & Scheidegger, *IER* (2022); Han, Yang & E, *QE* (2026); Fernández-Villaverde, Nuño & Perla, NBER WP 33117 (2024).

Where AI Enters (1), Continued: Reinforcement Learning



- **Critic**, target $\bar{\phi}$ held fixed:

$$\min_{\phi} \mathbb{E}[(u(c) + \beta V_{\bar{\phi}}(s') - V_{\phi}(s))^2]$$

- **Actor**, along simulated paths:

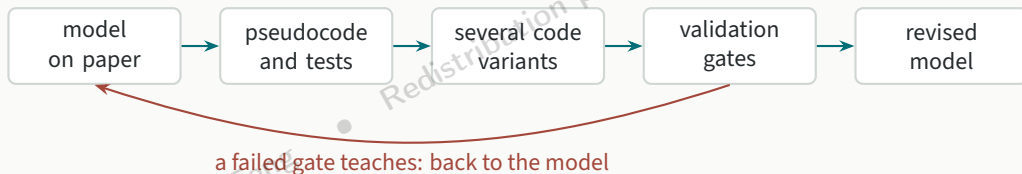
$$\max_{\theta} \mathbb{E}[u(c_{\theta}) + \beta V_{\phi}(s'_{\theta})]$$

- No transition law is written down: expectations are simulated averages.
- **Equilibrium: every household uses the trained rule. Sessions 3 and 4 build it.**

Source: Sutton & Barto, *Reinforcement Learning* (2018), ch. 13; Miao & Feng, *AI for Economists*, ch. 11.

Where AI Enters (2) and (3): Data, and a Collaborator

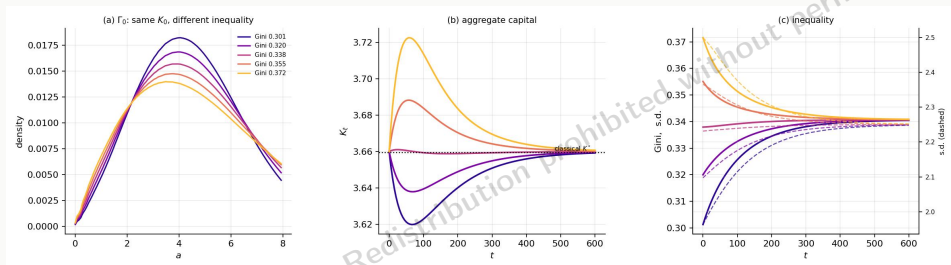
- ▶ **(2) Unstructured data become measurement.** Central-bank text gives a policy-stance index, night lights proxy for output, newspapers yield an uncertainty index.
- ▶ **(3) AI as a collaborator** helps you understand and write models, design algorithms, and implement code. Session 6 develops this loop.



- ▶ **AI writes variants quickly. You choose the state, the equilibrium concept, the benchmark, and what a failure means.**

Source: Hansen, McMahon & Prat (2018); Henderson, Storeygard & Weil (2012); Baker, Bloom & Davis (2016).

Research in Practice: Every Start Ends in the Same Place



- Five Aiyagari economies: the same $K_0 = 3.660$, Gini 0.301 to 0.372. All reach the classical $K^* = 3.6595$.
- A network trained on $\Gamma_{t+1} = \Phi(\Gamma_t)$ makes convergence a result, not an assumption.
- **Same K_0 , yet the distributions move for hundreds of periods. K alone misses it.**

Source: Feng, Han, Sargent & Zhu (2026), work in progress. Terminal K agrees to 3.5×10^{-8} .

What Stays With the Researcher

- ▶ A converged training loss certifies nothing. A solution is certified by a ladder of checks:
 - **Feasibility**: every choice respects the budget and borrowing constraints.
 - **Residuals on fresh draws**: unit-free Euler and Bellman errors, on states not used in training.
 - **A classical benchmark**: value function iteration on the low-dimensional case, where it still works.
 - **Theory**: policies are monotone, values concave, consumption smooth.
- ▶ The top rung is not computational: **your name on the result.**
- ▶ Today's lab climbs the first three rungs on the growth model, in under a minute.
- ▶ **Verification, judgment and responsibility stay with the researcher.**

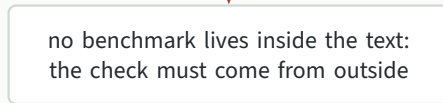
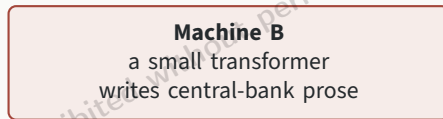
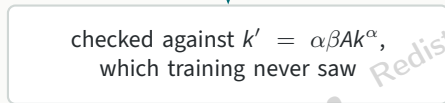
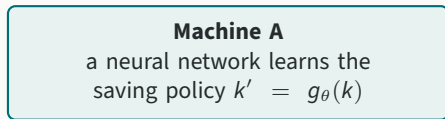
Source: Feng, "AI for Economics and the Economics of AI", working paper (2026), §3.3; Judd, Maliar & Maliar (2017).

Where Each Model Returns

Session	Object	What changes
1	Orientation	AI as tool, as object, and as an agent with incentives
2	Deep learning	Functions become trainable; equations become losses
3	Reinforcement learning	Sequential decisions are learned from interaction
4	Heterogeneous agents	Distributions and equilibrium become state variables
5	Text and LLMs	Language becomes data, measurement, and generation
6	RAG and agents	Retrieval, tools, traces, and governance in one workflow

- ▶ The growth model returns in Sessions 2 and 3; Aiyagari, Krusell–Smith, OLG and optimal taxation in Session 4.
- ▶ **Next: a first lab that solves the growth model with a network, and checks it. Deck 1.5 sets up the workbench it runs on.**

Lab: Two Machines, One Environment

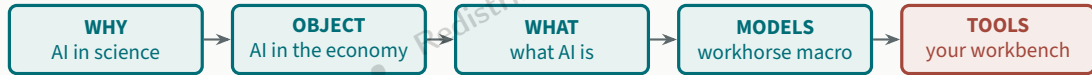


- ▶ Machine A minimises the squared Euler residual of Approach 2.
- ▶ Machine B learned the *look* of FOMC statements, not facts: no residual to compute.
- ▶ **Same technology, two epistemic positions: one output you can verify, one you cannot.**

Session 1.5

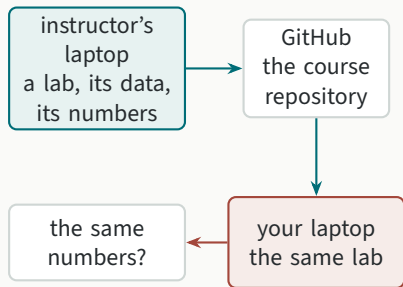
The Workbench: Python, VS Code, Git and GitHub

From a clean laptop to Python code that runs again, on any machine



Zhigang Feng

Why a Workbench



the workbench decides whether the
last arrow holds

- ▶ Every later session hands you a lab notebook that runs offline on a laptop CPU: no API key, no GPU.
- ▶ Whether it runs on *your* laptop depends on what you set up once: the course files, a Python, its packages, an editor, and a record of what you changed.
- ▶ This deck builds that, teaches the Python the labs are written in, and ends with the test: the Session 1 lab.
- ▶ **A result counts only if it runs again: tomorrow, on another machine, by someone else.**

Four Messages You Will Meet

what you see

`command not found: git`

`ModuleNotFoundError: No module named 'torch'`

`NameError: name 'beta' is not defined`

“It worked yesterday.”

what it means

the shell cannot find the program

the package is not in the Python running your code

a notebook depended on a cell it never reran

something changed, and nobody can say what

fixed in

GET

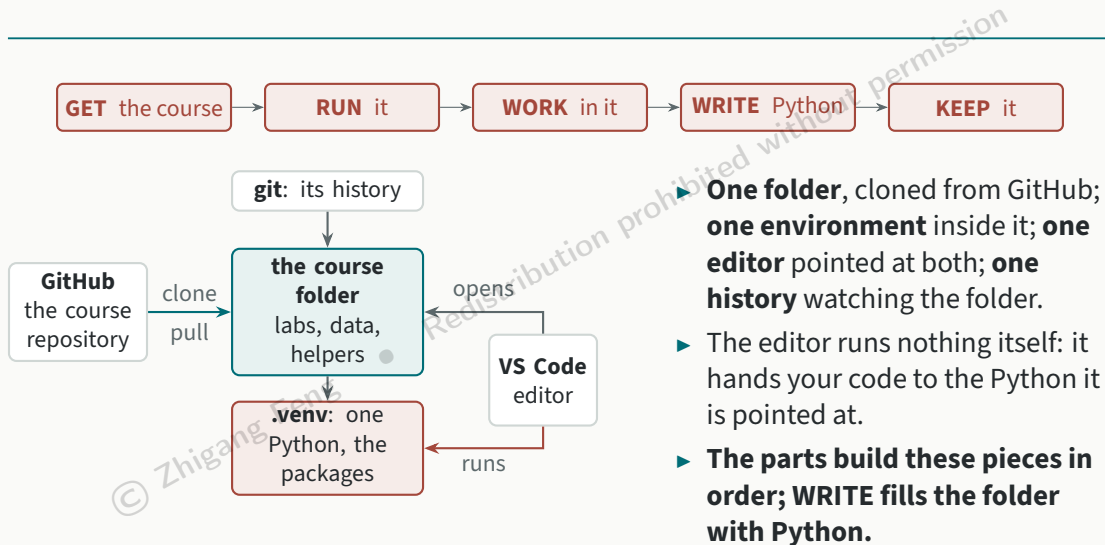
RUN, WORK

WORK

KEEP

- ▶ Four parts each remove one: GET the course, RUN it, WORK in it, KEEP it. A fifth, WRITE, teaches the Python itself.
- ▶ **None of the four is a Python problem; all four are workbench problems, and each costs more hours than the economics it blocks.**

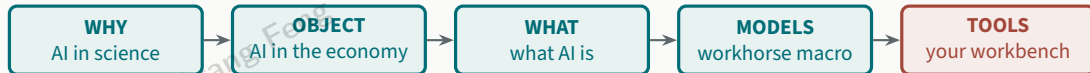
The Workbench in One Picture



Session 1.5

Part I • GET the Course

Three installs, a terminal, and one clone

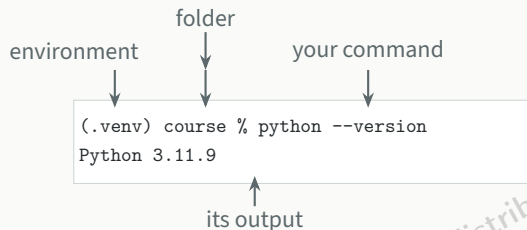


Three Installs, Two Operating Systems

	Windows	macOS
Python 3.11 the interpreter	the 3.11.9 installer from python.org; tick <i>Add python.exe to PATH</i> , off by default	the 3.11.9 installer from python.org; the command is <code>python3</code>
Git the history tool	Git for Windows, from git-scm.com	the command-line tools: <code>xcode-select --install</code>
VS Code the editor	code.visualstudio.com	code.visualstudio.com

- ▶ Three separate programs: installing VS Code installs neither Python nor git. If python.org is slow, the Huawei Cloud mirror carries the same installers.
- ▶ **Three installs, done once, before Session 2; check each with `--version`.**

The Terminal: Where Commands Go



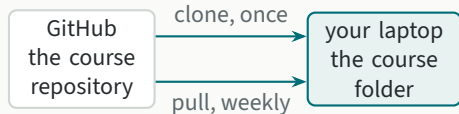
- ▶ A **shell** runs the commands you type: Terminal on macOS, PowerShell on Windows, or the terminal inside VS Code.
- ▶ `cd` enters a folder, `ls (dir)` lists it; `--version` asks a program which one it is.
- ▶ **PATH** is the list of folders searched for a program: `command not found` (“not recognized” on Windows) means it is on none of them.
- ▶ **Removes** `command not found`: **install, then open a new terminal.**

Clone the Course Once, Pull Every Week

```
git clone https://github.com/vonzhg/AI-ECON-2026.git
cd AI-ECON-2026
git pull
```

once

before each session



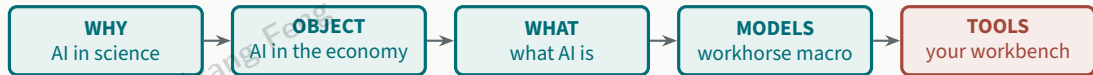
the lab reads helpers and data from the folders beside it

- ▶ Cloning a public repository needs no account. `git pull` brings each week's lab.
- ▶ A notebook downloaded on its own fails: it reads helpers and data from neighbouring folders.
- ▶ On a slow line, `git clone --depth 1` fetches only the latest version; `pull` still works.
- ▶ **The folder is the unit: clone it whole, update it with pull.**

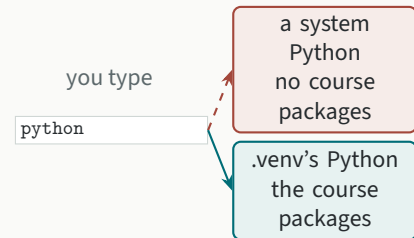
Session 1.5

Part II • RUN it

One Python, one environment, the course's packages



Which Python Is Running?



PATH is searched in order; activating
.venv puts its Python first

- ▶ A laptop can carry several Pythons. A command runs the first one found on PATH.
- ▶ `python --version` says which version; which python (where python on Windows) says which folder it lives in. Inside the course it should be the folder's `.venv`.
- ▶ The prompt tells you too: `(.venv)` at its start means the environment is active.
- ▶ **Run both commands before asking anyone for help: every answer depends on which Python.**

Environments: One Project, One Python, One Set of Versions

```
python3 -m venv .venv      # macOS: once, inside the course folder
source .venv/bin/activate  #      then in every new terminal
py -3.11 -m venv .venv     # Windows: once
.venv\Scripts\Activate.ps1 #      then in every new PowerShell
```

course folder
.venv: torch,
numpy 2

another
project
its own .venv:
numpy 1

two environments never share a
version, so they never fight

PowerShell refuses the script? Once:
Set-ExecutionPolicy RemoteSigned
-Scope CurrentUser

- ▶ An **environment** is a folder holding one Python and one set of packages for one project.
- ▶ Install *into* the active one with `python -m pip`. Broken? Delete `.venv`, create it again.
- ▶ **Most** `ModuleNotFoundError`s are a **package installed into a different Python**.

Install the Course's Packages, Once

```
python -m pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple # once
python -m pip install numpy pandas matplotlib scipy scikit-learn torch jupyter ipykernel
```

	S1	S2	S3	S4	S5	S6
numpy	●	●	●	●	●	●
matplotlib	●	●	●	●	●	●
torch	●	●	●	●	●	●
pandas	●	●	●	●	●	●
scikit-learn	●	●	●	●	●	●
scipy	●	●	●	●	●	●

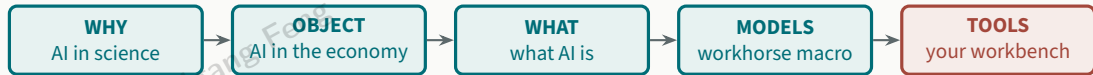
which Session lab imports what; jupyter
and ipykernel run the notebooks

- ▶ The first line points pip at the Tsinghua mirror; the second installs all eight packages into the active `.venv`.
- ▶ The list names packages, not versions: the lab's first cell prints the versions you got. `torch` (PyTorch) is installed now and taught in Deck 2.4.
- ▶ **Removes** `ModuleNotFoundError` **at its source: everything the six labs import is now installed, in the Python they will use.**

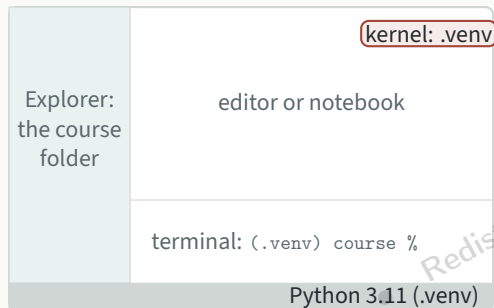
Session 1.5

Part III • WORK in It

VS Code, the kernel, and notebooks



VS Code Runs Nothing Itself



status bar: the interpreter the editor uses

- ▶ VS Code shows files and hands code to the Python you select. Install two extensions by Microsoft: *Python* and *Jupyter*.
- ▶ **File** → **Open Folder** on the course folder, not a single file; VS Code then finds its `.venv`.
- ▶ An AI assistant in the editor writes code; it does not run or check it (Deck 1.3; Session 6).
- ▶ **The editor is a window onto a folder and a Python. Get both right.**

Three Places Choose a Python

the terminal
prompt

`(.venv)`

the status bar

Python 3.11 `(.venv)`

the notebook
kernel

a system Python

one of three on the wrong Python: the
notebook cannot import torch

- ▶ All three must say `.venv`: the terminal you install in, the editor, and the notebook you run.
- ▶ **Python: Select Interpreter** sets the editor's; **Select Kernel** (a notebook's top right) sets the notebook's.
- ▶ A terminal opened inside VS Code activates the selected `.venv`; one opened elsewhere does not until you activate it.
- ▶ **Removes the other**
`ModuleNotFoundError: installed, but not where the notebook looks.`

A Notebook Is Not a Program

on screen, as you ran it

[5] `V = solve(beta)`

[2] `beta = 0.96`



Restart, then Run All

[1] `V = solve(beta)`

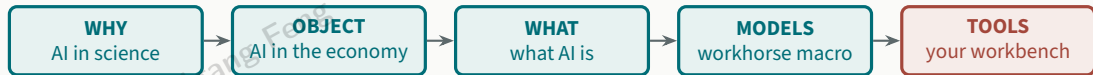
`NameError: name 'beta' is not defined`

- ▶ A notebook keeps every variable until its kernel restarts, even ones whose cell you edited or deleted; the numbers in brackets record the order you ran things.
- ▶ So a notebook can look right and depend on a state nobody can rebuild.
- ▶ Before you trust or send a number: **Restart, then Run All.** A `NameError` then has found the hidden dependency.
- ▶ **Long or repeated runs go in a script that reads its settings at the top and writes results to disk.**

Session 1.5

Part IV • WRITE Python

The language the labs are written in, on the lab's growth model



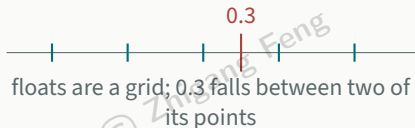
Values, Names and Types

```
>>> alpha = 0.33          # capital share
>>> beta = 0.96           # discount factor
>>> A = 1.0               # productivity
>>> k_ss = (alpha*beta*A) ** (1/(1-alpha))
>>> k_ss
0.17984701877776363
>>> type(alpha), type(200)
(<class 'float'>, <class 'int'>)
>>> print(f"steady state: {k_ss:.4f}")
steady state: 0.1798
alpha → 0.33      beta → 0.96      k_ss → 0.1798...
```

- ▶ The lab's growth model:
 $k' = \alpha\beta Ak^\alpha$, steady state
 $k^* = (\alpha\beta A)^{1/(1-\alpha)}$.
- ▶ = names a value; it is not an equation. ** is a power.
- ▶ Types: int, float, text (str), True/False (bool); Python works them out.
- ▶ **A name points to a value; an f-string prints it.**

Numbers Are Not Exact

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True
>>> 1 / 2, 1 // 2, 7 // 2
(0.5, 0, 3)
```



- ▶ A float carries about 16 significant digits, and most decimals are not exactly on its grid.
- ▶ Never test a computed float with `==`; compare with a tolerance, as `math.isclose` does.
- ▶ `//` divides and rounds down to an integer: how you turn a value into a grid position.
- ▶ **Sessions 2–4 judge a solution by residuals near 10^{-6} : know that 10^{-16} is the floor.**

Lists: A Path of Capital

```
>>> path = [0.5 * k_ss]          # start at k*/2
>>> for t in range(5):
...     path.append(alpha*beta*A*path[-1]**alpha)
...
>>> len(path)
6
>>> [round(k, 4) for k in path]
[0.0899, 0.1431, 0.1668, 0.1754, 0.1784, 0.1794]
>>> [round(k, 4) for k in path[1:3]]
[0.1431, 0.1668]
```



- ▶ A list keeps values in order; `append` adds at the end; `path[-1]` is the last.
- ▶ Positions start at 0; `[1:3]` stops *before* 3.
- ▶ `[f(x) for x in xs]` builds a list in one line. **An off-by-one index raises no error.**

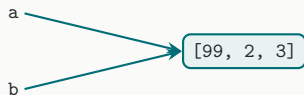
Dictionaries and Tuples: Parameters With Names

```
>>> params = dict(alpha=0.33, beta=0.96, A=1.0)
>>> params["beta"]
0.96
>>> list(params.keys())
['alpha', 'beta', 'A']
>>> s = params["alpha"] * params["beta"]
>>> y = params["A"] * k_ss ** params["alpha"]
>>> k_next, c = s * y, (1 - s) * y
>>> round(k_next, 4), round(c, 4)
(0.1798, 0.3879)
```



- ▶ A **dict** maps names to values: a calibration as one object, read by name.
- ▶ A **tuple** holds several values; `k_next, c = ...` unpacks it.
- ▶ At k^* the policy returns k^* (0.1798): a check.
- ▶ **Name parameters in a dict, not in your memory.**

A Name Is Not a Copy



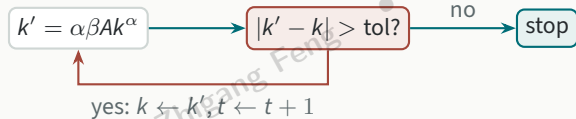
`b = a`: a second name; `c = a.copy()`: a second object

```
>>> a = [1, 2, 3]; b = a; b[0] = 99
>>> a
[99, 2, 3]
```

- ▶ Assignment never copies: `b = a` puts a second name on the same object. NumPy slices are *views* of the same memory too.
- ▶ **Value function iteration** repeats $V \leftarrow T(V)$ until the change $\max |V - V_{\text{old}}|$ is tiny.
- ▶ Write `V_old = v` and update `v` in place: both names see the update, the change is always 0, and the loop reports convergence after one step.
- ▶ **Write `V_old = V.copy()`: the bug is silent, the fix is one word.**

Control Flow: Iterate to the Steady State

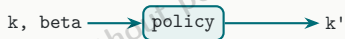
```
>>> k, t, tol = 0.5 * k_ss, 0, 1e-8
>>> k_next = alpha*beta*A * k**alpha
>>> while abs(k_next - k) > tol:
...     k, t = k_next, t + 1
...     k_next = alpha*beta*A * k**alpha
...
>>> print(t, round(k_next, 6), round(k_ss, 6))
15 0.179847 0.179847
```



- ▶ for `t in range(5)` repeats a known number of times; `while` repeats until its test fails.
- ▶ The indented lines are the block that repeats; indentation is the syntax.
- ▶ From $k^*/2$ the loop stops after 15 steps, at k^* to six decimals.
- ▶ **Every solver in this course stops this way: on a tolerance.**

Functions: Name the Rule Once

```
>>> def policy(k, alpha=0.33, beta=0.96, A=1.0):  
...     "Next capital: alpha*beta*A*k**alpha."  
...     return alpha * beta * A * k ** alpha  
...  
>>> round(policy(0.1), 4)  
0.1482  
>>> round(policy(0.1, beta=0.99), 4)  
0.1528  
>>> def step(k):  
...     k_next = policy(k)  
...     return k_next, k ** alpha - k_next  
...  
>>> [round(x, 4) for x in step(k_ss)]  
[0.1798, 0.3879]
```



- ▶ def names a rule; return sends back the result; the string says what it does.
- ▶ Defaults (beta=0.96) change by name: policy(0.1, beta=0.99).
- ▶ A function can return a tuple: k' and consumption.
- ▶ **One rule, one definition, many calls.**

Modules and import

```
# growth.py, a file beside your notebook
ALPHA, BETA, A = 0.33, 0.96, 1.0
```

```
def policy(k):
    """Next period's capital."""
    return ALPHA * BETA * A * k ** ALPHA

if __name__ == "__main__": # only as a script
    print(round(policy(0.1), 4))

>>> from growth import policy
>>> round(policy(0.1), 4)
0.1482
```



- ▶ A **module** is a file of Python; `import` runs it once and lends you its names. NumPy is a module: `import numpy as np`.
- ▶ The last two lines run only when the file is run as a script (`python growth.py`), not on `import`.
- ▶ **The labs import helpers from their folder: that is why you clone it whole.**

A Model as a Class

```
class GrowthModel:
    def __init__(self, alpha=0.33, beta=0.96, A=1.0):
        self.alpha, self.beta, self.A = alpha, beta, A
        self.k_ss = (alpha*beta*A) ** (1/(1-alpha))

    def policy(self, k):
        return self.alpha*self.beta*self.A*k**self.alpha

    def simulate(self, k0, T):
        path = [k0]
        for _ in range(T):
            path.append(self.policy(path[-1]))
        return path
```

```
>>> m = GrowthModel()
>>> round(m.k_ss, 4)
0.1798
>>> path = m.simulate(0.05, 30)
>>> round(path[-1], 4)
0.1798
```

- ▶ A **class** bundles parameters (self.alpha) with methods.
- ▶ `__init__` runs when you make one: `GrowthModel(beta=0.99)` is another economy.
- ▶ **The lab's networks are classes like this.**

Reading an Error

```
>>> policy("0.2")
Traceback (most recent call last):
  File "<console>", line 1, in <module>
    File "<console>", line 3, in policy
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'float'
>>> params["Beta"]
Traceback (most recent call last):
  ...
KeyError: 'Beta'
```

above: where, in your code

last line: what went wrong

- ▶ **NameError: never defined. TypeError: wrong kind of value. IndexError: past the end. KeyError: not in the dict.**
- ▶ **Read the last line first; paste the whole message when you ask for help.**

NumPy: One Array, No Loop

```
>>> import numpy as np
>>> k = np.linspace(0.2*k_ss, 2*k_ss, 200)
>>> k.shape, k.dtype
((200,), dtype('float64'))
>>> k_next = alpha*beta*A * k**alpha
>>> k_next[:3].round(4)
array([0.1057, 0.1073, 0.1088])
>>> loop = [policy(x) for x in k]
>>> np.allclose(k_next, loop)
True
```



one block of 200 float64 numbers: a grid on k

- ▶ An **array** holds numbers of one type in one block of memory. `linspace` makes a grid of 200 points from $0.2k^*$ to $2k^*$.
- ▶ Arithmetic on an array applies to every element: one line computes the policy at all 200 points.
- ▶ The list loop gives the same numbers; NumPy runs its loop in compiled code, far faster.
- ▶ **On a grid, write the formula once and let the array carry it.**

Shapes and Broadcasting

```
>>> a = np.ones(3)           # shape (3,)
>>> b = np.ones((3, 1))      # shape (3, 1)
>>> (a + b).shape
(3, 3)
>>> d = k[:, None] - k[None, :]
>>> d.shape
(200, 200)
>>> assert k_next.shape == k.shape
```

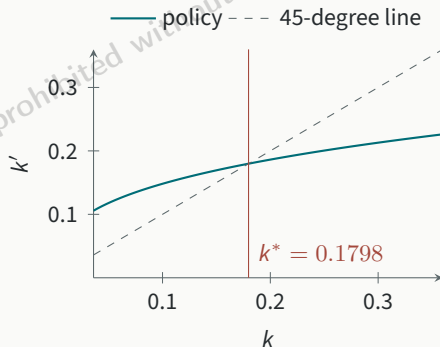


- **Broadcasting:** an axis of length 1 is stretched to match. $k[:, \text{None}] - k[\text{None}, :]$ is every $k - k'$ on the grid at once, the matrix Session 2's solvers search.
- The trap: shapes (3,) and (3,1) give a (3,3) table, not three sums, and no error.
- PyTorch's tensors follow the same rules (Deck 2.4).
- **Assert the shape you expect.**

Plot It: The Policy Against the 45-Degree Line

```
import matplotlib.pyplot as plt
plt.plot(k, k_next, label="policy")
plt.plot(k, k, "--", label="45-degree line")
plt.axvline(k_ss, color="grey")
plt.xlabel("k"); plt.ylabel("k'")
plt.legend()
plt.savefig("policy.pdf")    # before show()
plt.show()
```

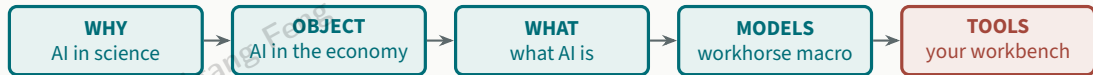
- ▶ One line per curve; `savefig` writes the file a paper uses.
- ▶ **A figure your code can redraw is a result; a screenshot is not.**



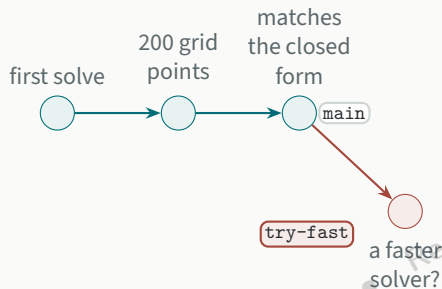
Session 1.5

Part V • KEEP it

A history of your work, and a copy on GitHub

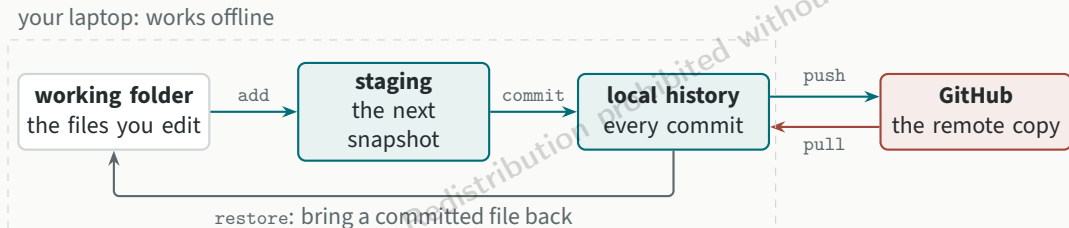


Git: A History of Your Project



- ▶ **Git** keeps the history of a project folder. A **commit** is a snapshot of every tracked file, with an author, a time and a message.
- ▶ The **history** is the chain of commits; any of them can be compared with today or brought back. A **branch** is a line of commits you may throw away.
- ▶ Git is not GitHub: git runs offline; GitHub keeps a copy online (Chacon & Straub, *Pro Git*, 2014).
- ▶ **Removes “it worked yesterday”: every committed state can be recovered.**

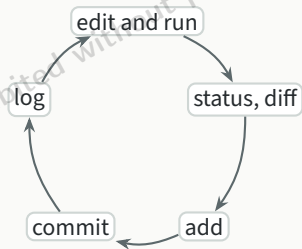
Three Places on Your Machine, One on GitHub



- ▶ You edit in the working folder; `add` chooses which changes go into the next snapshot; `commit` writes it into the history; `push` and `pull` exchange with GitHub.
- ▶ **A commit is local until you push: until then it lives on one disk only.**

The Daily Loop

```
git config --global user.name "Your Name"    # once
git config --global user.email "you@uni.edu"
git status      # what changed?
git diff        # how, line by line?
git add run.py
git commit -m "VFI on 200 points: max Euler error 1.2e-4"
git log --oneline # the story so far
```



- ▶ Tell git who you are once; then loop: look at what changed, choose it, commit it with a message.
- ▶ **Commit when a result changes, and put the result in the message: it is a note to yourself in three months.**

When git pull Refuses

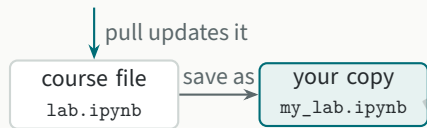
```
$ git pull
```

```
error: Your local changes to the following files would be overwritten by merge:
```

```
lab.ipynb
```

```
Please commit your changes or stash them before you merge.
```

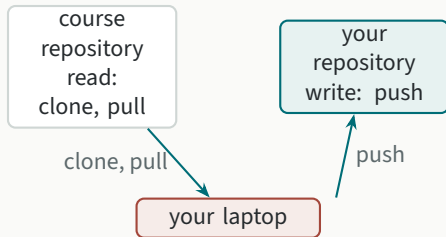
```
Aborting
```



run and edit your copy; leave the
course file to pull

- ▶ Running a notebook saves its outputs into the file, so git sees a change you never typed, and the next pull stops rather than overwrite it.
- ▶ To keep your run: save it under your own name first. To discard it: `git restore lab.ipynb`, then pull.
- ▶ **Run your own copy; let pull own the course's files.**

GitHub: Your Own Repository



one laptop, two remotes: one you read,
one you write

- ▶ Make a GitHub account; create a repository for your own work and clone it, the verb from Part I.
- ▶ Pushing needs a sign-in, and GitHub has not accepted account passwords for git since 2021: use an SSH key, or sign in once with GitHub's `gh auth login`.
- ▶ If a laptop install fails, GitHub Codespaces runs the same notebooks in a browser.
- ▶ **Read from the course, write to your own.**

What Never Goes Into a Repository

commit

code
the package list
a README: how
to run it

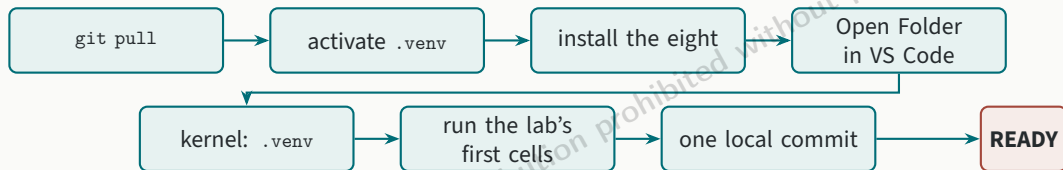
```
# .gitignore
.venv/
*.pt
.env
```

never

data you do not
own
model
checkpoints
keys, passwords
.venv

- ▶ A key pushed once is public, and stays in the history after you delete the file.
- ▶ A `.gitignore` lists what git must never track; write it once, at the start.
- ▶ What always goes in is what someone else needs to rerun you: code, the package list, a README.
- ▶ **In Session 6 git is also the safety net under an AI agent: the habits start here.**

Ready: The Check Before Session 2



- ▶ The lab's first cells print Python, every package's version and git, then find the course folder; a missing piece is named, with what to install.
- ▶ A commit of any small file in your own repository shows that git works; it needs no account.
- ▶ **If every step prints, the rest of the course runs on your laptop.**

Open the Lab, and the Bridge to Session 2

- ▶ Open the Session 1 lab notebook in the course folder, select the `.venv` kernel, and run the cells from the top. Its first cells are the check on the last slide.
- ▶ Two models, both offline on a laptop CPU with fixed seeds: a neural network that learns the growth model's saving policy and is checked against the closed form, and a small text generator trained on FOMC statements, with no benchmark inside its text.
- ▶ Fifteen minutes: the check, the two models, then three temperatures (0.5, 0.8, 1.2).
- ▶ Note privately what changed when you turned the dial: fluency, repetition, reliability.
- ▶ **Session 2 opens the first model: from one neuron to a trained network. PyTorch, NumPy's arrays plus derivatives computed for you, is Deck 2.4.**